

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

General Instructions — CS3....

(5) Mod-01 Lec-17 TYPES O...

C-shell project skeleton code

Share

Give the heavy skeleton of part D, also explain how to do the remaining part too

Checking existing headers to align the skeleton with the codebase.

Let me read the pieces Part D has to plug into — your existing exec path, the command/job data types, the shell state, and the main loop — so the skeleton compiles against your actual code and calls your real functions rather than inventing new ones.

Ran 2 commands

This is the real IIIT-H CS3.301 spec, so I've mapped the skeleton onto your actual `d_jobs.c` structure and the `exec_*` API it builds on. Let me pull the mechanical helper bodies (job table, async-safe printing, name building) so I can hand those to you complete and leave only the Part-D learning objectives as your work.

Ran 2 commands

Last piece — the foreground dispatch and the D1 sequencer, so the skeleton's control flow and my explanation of "command not found stops the sequence" (D1 rule 3) are exactly right.

Ran a command

I've now traced the whole module — the mechanical helpers, the `exec_*` /builtin APIs it plugs into, and exactly where the Part-D concepts live. Here's the heavy skeleton. Everything mechanical is written and compiles clean under your flags; the five spots that *are* the Part-D learning are left as clearly-specified `TODO` s.

Ran 2 commands, created a file

One expected hiccup: `wstr` / `wuint` are the async-safe printers you'll call in `TODO-1`, so with that `TODO` unfilled they read as "defined but unused" under `-Werror` . I'll mark them so the skeleton builds cleanly now and they're ready for you to use.

Edited a file, ran a command

Builds clean under `-Werror` and links; the binary runs (foreground sequencing is intentionally

Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

Here's how to finish each `TODO`. Do them in this order — each one makes the next testable — and note the whole thing hinges on one idea: **a pipeline is one job, which is one process group**.

1. TODO-3 — process groups (do this first). Every stage of a pipeline joins a single group whose id (`pgid`) is the first stage's pid. Each child calls `setpgid` to join, and the parent calls it again for the same child; you do it on both sides because after `fork()` you don't control whether parent or child runs first, and the child might even `exec` before the parent gets there — whichever call lands first wins, the other is a harmless no-op. Once groups exist, the group-`wait` (`-pgid`) and the terminal handoff (`tcsetpgrp`) work, and background jobs stop stealing the terminal (spec #12). Test: run `sleep 50 &`, then in another terminal `ps -o pid,pgid,cmd` — the `sleep`'s `pgid` should equal its own pid and differ from the shell's.

2. TODO-2 — `wait_group` (foreground wait). Loop, waiting on the whole group (negative `pgid`) with `WUNTRACED`, counting down the alive members until they're all gone. Two traps: treat `errno == EINTR` as "a handler ran, just retry" rather than an error; and wait on the *group*, not a single pid — the classic bug is waiting on one pid so a 3-stage pipe only waits for one stage and the prompt races the output. Test: `echo hi | cat | cat` must print `hi` and only *then* prompt; `sleep 3` should freeze the prompt for 3 seconds.

3. TODO-1 — `handle_dead` (background completion message). This runs *inside* the `SIGCHLD` handler, so it must be async-signal-safe: build the message with the provided `wstr/wuint`, never `printf/fprintf` (calling those from a handler is the single most common crash in this part). Only print when the *last* member of a pipeline dies — that's what `nalive` tracks — choose "normally" vs "abnormally" from whether it was killed by a signal, and report the `pgid` as the pid (spec #13). Test: `sleep 2 &`, then just sit there — about 2s later you should see `sleep with pid <n> exited normally` appear even at an idle prompt (#10). `sleep 100 &` then `kill -9 <pid>` → `exited abnormally`.

4. TODO-4 — the `[job] pid` line. In the background branch, print the bracketed job number and pid before returning, and *don't* wait. Test: `sleep 5 &` prints `[1] <pid>` and gives you a prompt instantly; launch several and the numbers only ever climb, never repeat (#4).

5. TODO-5 — D1 rule 3. Before launching a single external command, resolve it with `exec_resolve`; if it can't be resolved, print the not-found line and return `RUN_FAILED`, which is what makes `jobs_run_sequence` stop the rest of the `;` chain. The boundary to get right: a command that *does* start and then exits non-zero has **not** failed (rule 4) — the sequence continues — so gate only on "couldn't start," never on exit status. Test: `echo a ; nope ; echo b` prints `a`, the not-found line, and stops (no `b`); `false ; echo b` still prints `b`.

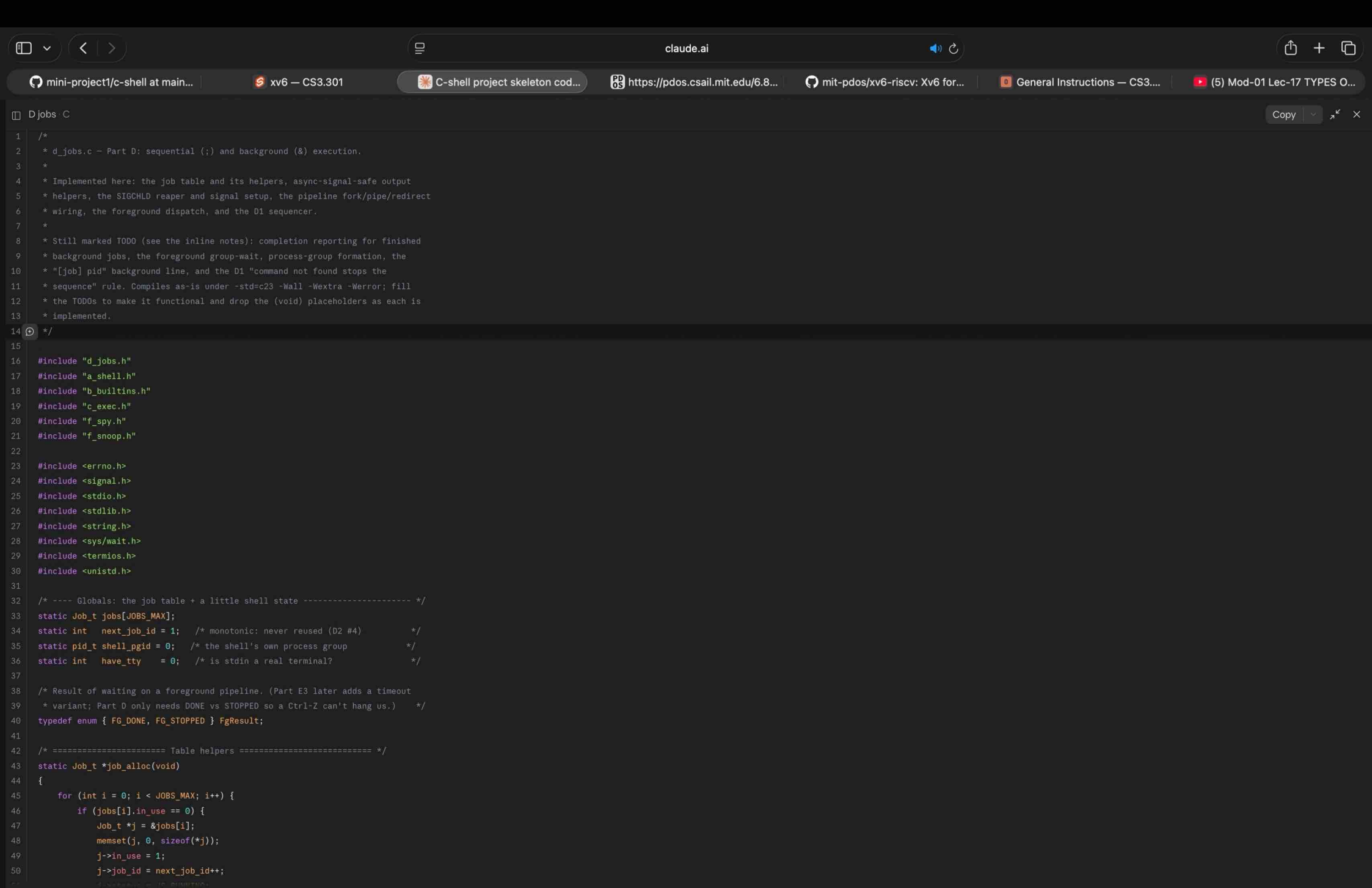
Two things already handled in the plumbing that are worth understanding rather than rewriting: the `sigchld_block/unblock` around `launch_pipeline` is what defers a background completion message until a running foreground job finishes (spec #11) — that's why the block is there. And the `FG_STOPPED` branch plus the empty `jobs_activities/jobs_resume/jobs_ping` strings are the seams where **Part E** (Ctrl-Z, `fg/bg`, `activities`, `ping`) drops in without restructuring anything.

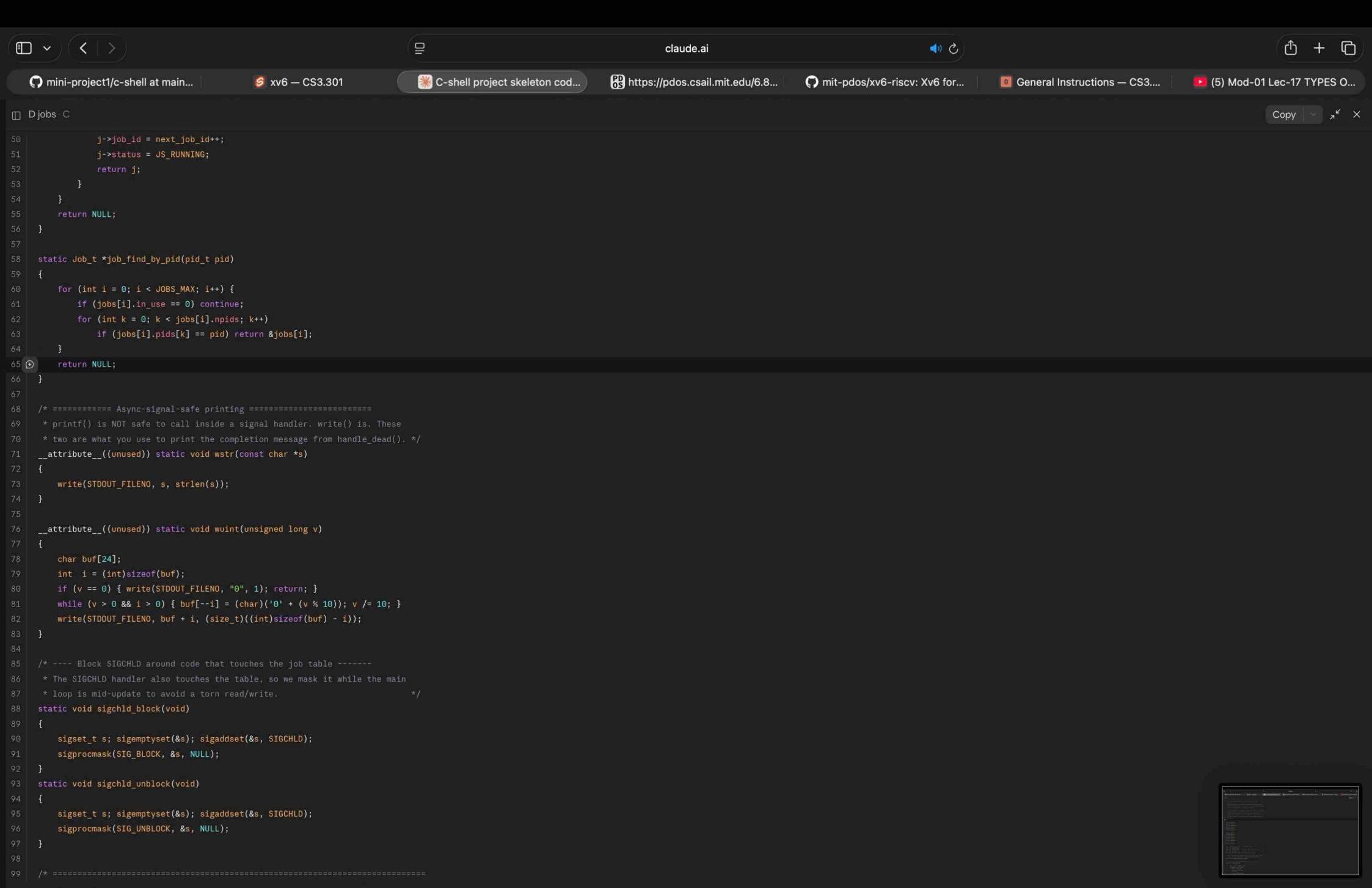
Say "give the heavy skeleton of part E" (or F) and I'll build those the same way.

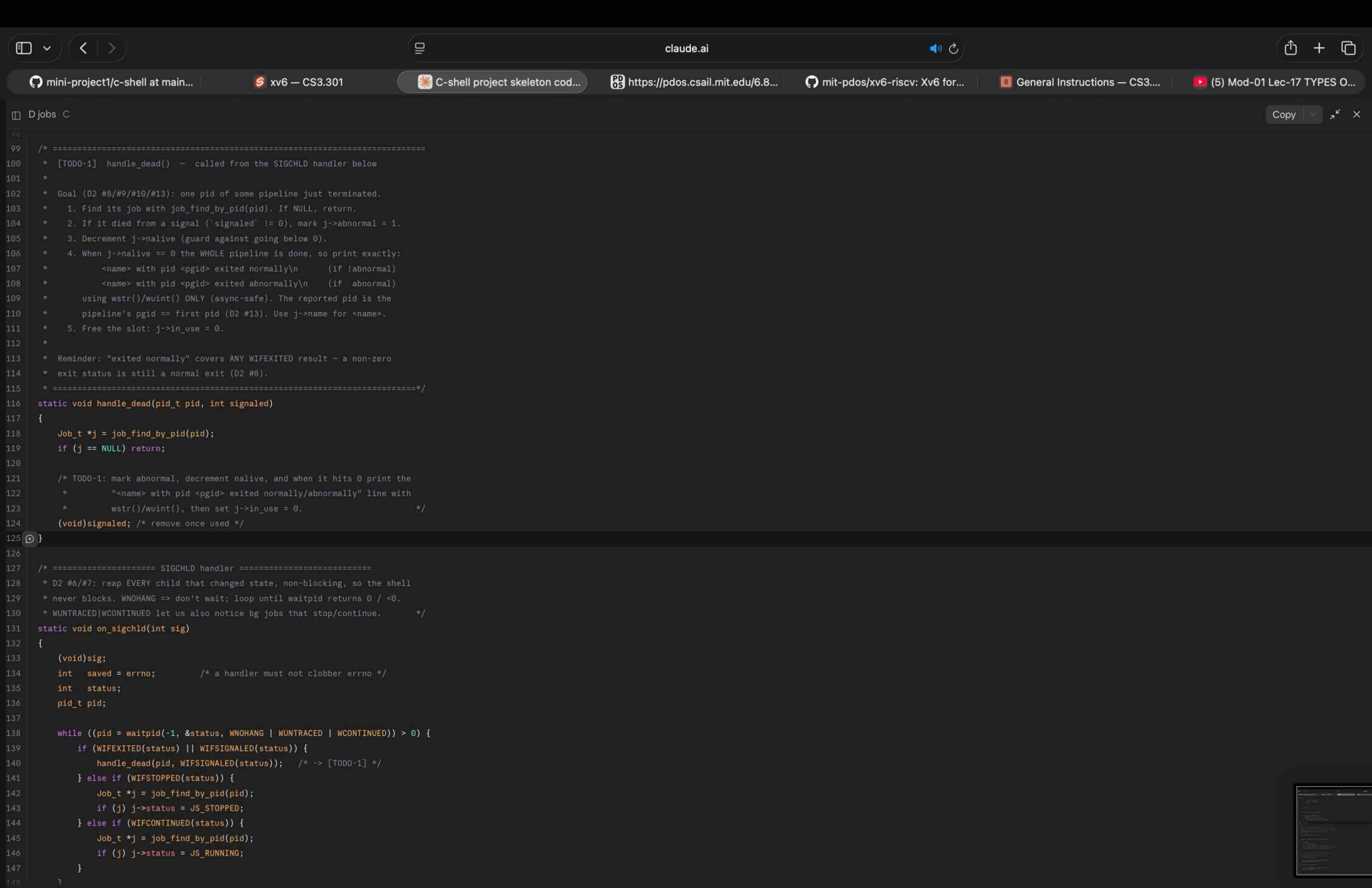


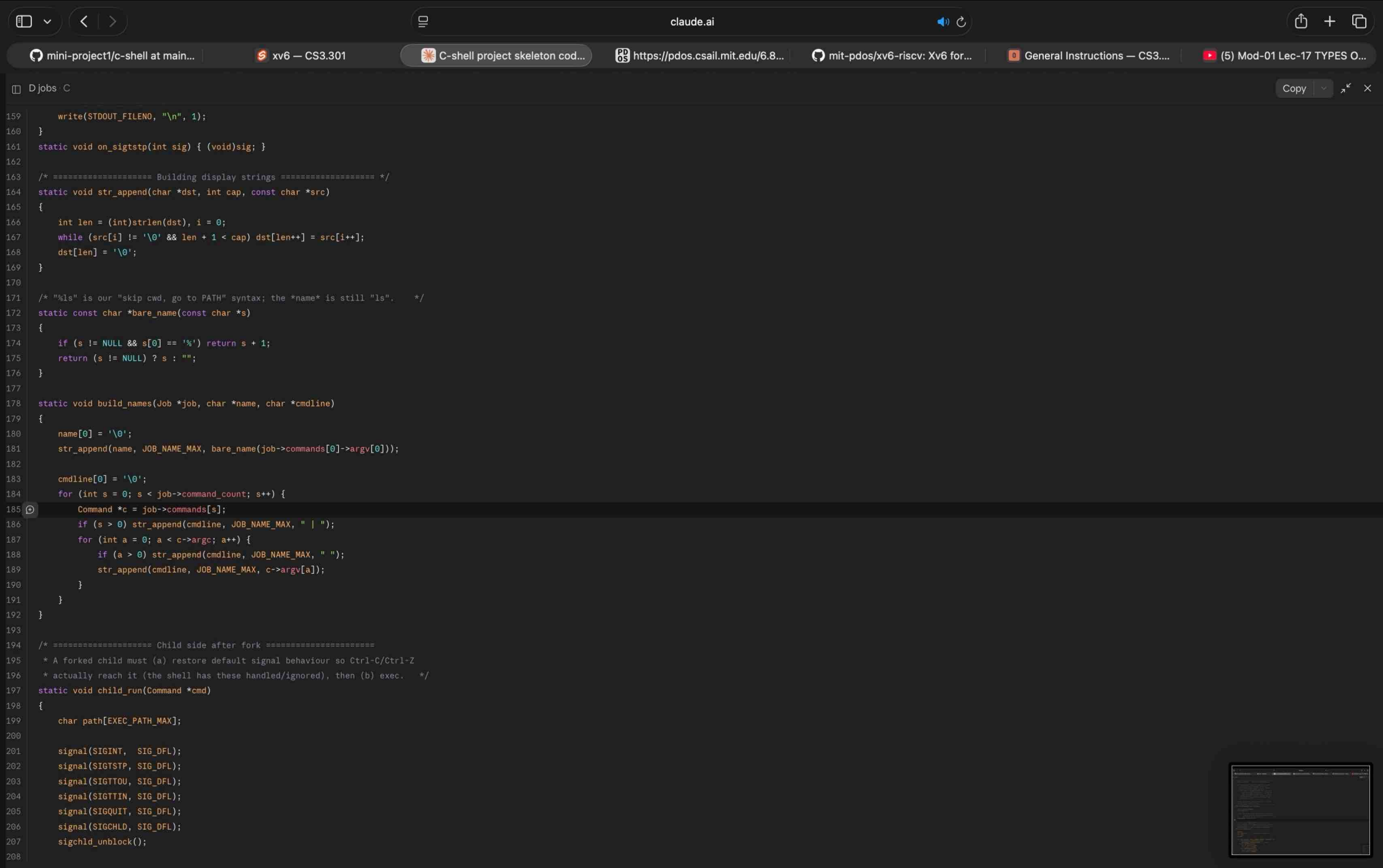
Opus 4.8 High

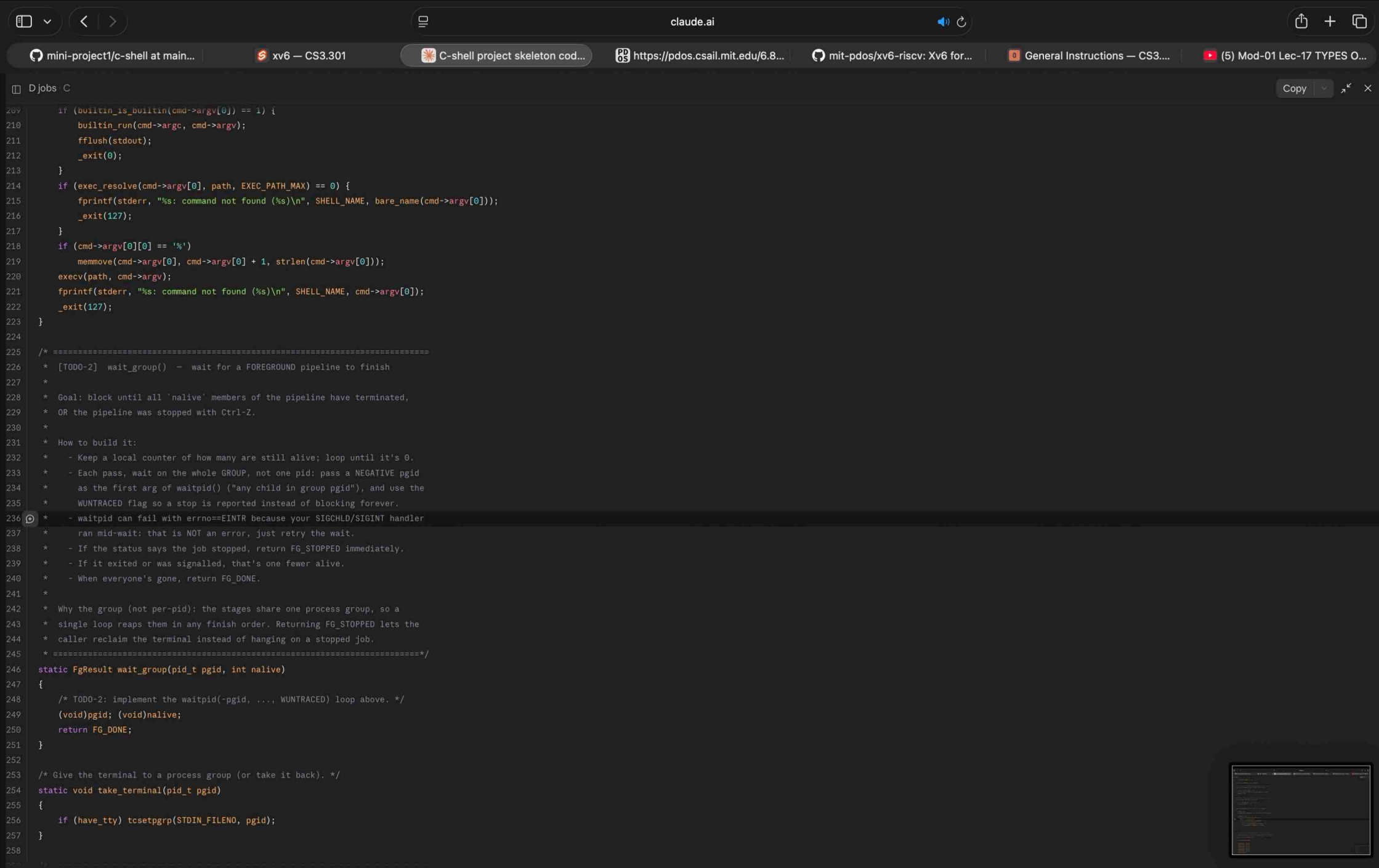
Copy











claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

General Instructions — CS3....

(5) Mod-01 Lec-17 TYPES O...

Djobs C

Copy

```
253  /* Give the terminal to a process group (or take it back). */
254  static void take_terminal(pid_t pgid)
255  {
256      if (have_tty) tcsetpgrp(STDIN_FILENO, pgid);
257  }
258
259  /* =====
260   * launch_pipeline() - fork every stage of ONE pipeline, wired for pipes
261   *                    and redirection. The pipe/fork/dup2 plumbing is DONE.
262   *                    You fill [TODO-3] and [TODO-4].
263   * =====*/
264  static void launch_pipeline(Job *job, int background)
265  {
266      Redirection redirs[JOB_MAX_PIDS];
267      int  opened[JOB_MAX_PIDS];
268      int  stages  = job->command_count;
269      int  prev_read = -1;
270      pid_t pgid = 0;
271      pid_t plist[JOB_MAX_PIDS];
272      char pnm[JOB_MAX_PIDS][JOB_PNAME_MAX];
273      int  npids = 0;
274      char name[JOB_NAME_MAX], cmdline[JOB_NAME_MAX];
275
276      if (stages > JOB_MAX_PIDS) stages = JOB_MAX_PIDS;
277      build_names(job, name, cmdline);
278
279      sigchld_block(); /* keep the table stable while we build the job */
280
281      for (int i = 0; i < stages; i++) {
282          Command *cmd = job->commands[i];
283          int  pipe_fds[2];
284          int  has_pipe = 0;
285          pid_t pid;
286
287          opened[i] = 0;
288
289          if (i < stages - 1) { /* need a pipe to the next stage */
290              if (pipe(pipe_fds) < 0) { perror("cshell: pipe"); break; }
291              has_pipe = 1;
292          }
293          if (cmd->argc == 0 || redir_open(cmd, &redirs[i]) == 0) {
294              if (prev_read >= 0) close(prev_read);
295              prev_read = has_pipe ? (close(pipe_fds[1]), pipe_fds[0]) : -1;
296              continue;
297          }
298          opened[i] = 1;
299
300          fflush(stdout);
301          pid = fork();
302          if (pid < 0) { perror("cshell: fork"); break; }
```

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

General Instructions — CS3....

(5) Mod-01 Lec-17 TYPES O...

D jobs C

Copy

```
302     if (pid < 0) { perror("cs-shell: fork"); break; }
303
304     if (pid == 0) { /* ----- CHILD ----- */
305         /* [TODO-3a] Put this child in the pipeline's process group with
306          *   setpgid(). Pass 0 as the pid arg (means "me") and the group
307          *   leader's pgid as the group. For the FIRST stage pgid is still
308          *   0, which conveniently makes that child its own group leader -
309          *   which is what you want (pgid == first pid, D2 #13). */
310
311         if (prev_read >= 0) { /* stdin from previous pipe */
312             dup2(prev_read, STDIN_FILENO);
313             close(prev_read);
314         }
315         if (has_pipe) { /* stdout into next pipe */
316             close(pipe_fds[0]);
317             dup2(pipe_fds[1], STDOUT_FILENO);
318             close(pipe_fds[1]);
319         }
320         redir_apply(&redirs[i]); /* < > >> override pipes */
321         for (int t = 0; t < redirs[i].target_count; t++)
322             close(redirs[i].targets[t]);
323         child_run(cmd); /* never returns */
324     }
325
326     /* ----- PARENT ----- */
327     /* [TODO-3b] On the FIRST child, remember its pid as the group's pgid.
328      *   Then call setpgid() from the PARENT too, to place this child in the
329      *   group. Doing it on both sides is deliberate: after fork() you don't
330      *   know whether parent or child runs first, and the child may even
331      *   exec before you get here - whichever setpgid lands first wins, the
332      *   other is a harmless no-op. Skip this and background jobs stay in the
333      *   shell's group and fight for the terminal (breaks D2 #12). */
334
335     plist[npids] = pid;
336     strncpy(pnm[npids], bare_name(cmd->argv[0]), JOB_PNAME_MAX - 1);
337     pnm[npids][JOB_PNAME_MAX - 1] = '\0';
338     npids++;
339
340     if (prev_read >= 0) close(prev_read);
341     prev_read = has_pipe ? (close(pipe_fds[1]), pipe_fds[0]) : -1;
342 }
343 if (prev_read >= 0) close(prev_read);
344
345 /* ----- BACKGROUND ----- */
346 if (background == 1) {
347     Job_t *slot = job_alloc();
348     if (slot != NULL) {
349         slot->pgid = pgid;
350         slot->npids = npids;
351         slot->nalive = npids;
352         slot->status = JS_RUNNING;
```

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

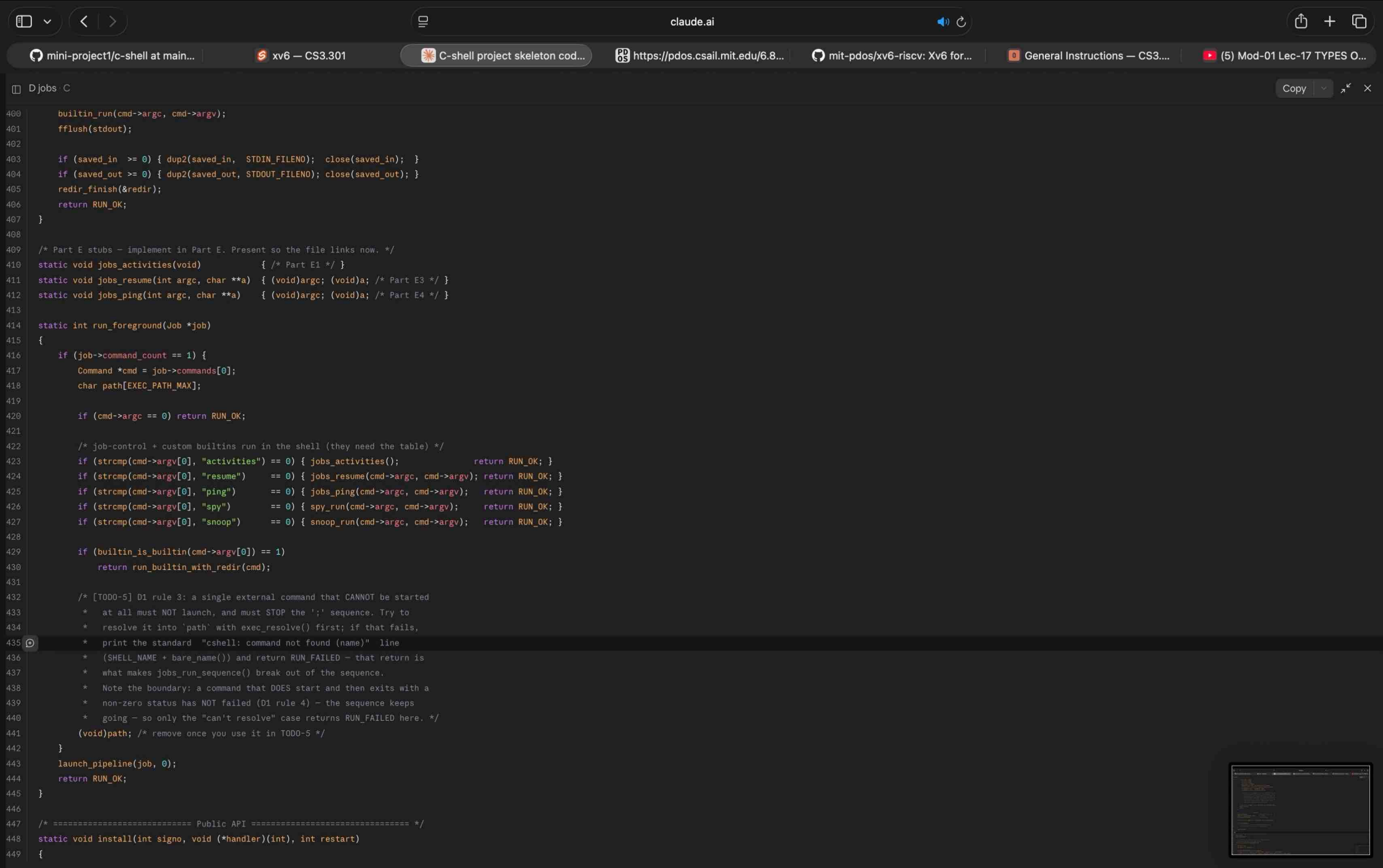
General Instructions — CS3....

(5) Mod-01 Lec-17 TYPES O...

D jobs C

Copy

```
349     slot->pgid = pgid;
350     slot->npids = npids;
351     slot->nalive = npids;
352     slot->status = JS_RUNNING;
353     memcpy(slot->pids, plist, sizeof(pid_t) * (size_t)npids);
354     for (int i = 0; i < npids; i++) strcpy(slot->pnames[i], pnm[i]);
355     str_append(slot->name, JOB_NAME_MAX, name);
356     str_append(slot->cmdline, JOB_NAME_MAX, cmdline);
357
358     /* [TODO-4] Print the background job line here, BEFORE returning and
359     * before any command output (D2 #3). Format is bracketed job id,
360     * a space, then the pid - match the spec example "[1] 4021" - and
361     * the pid to report is the group's pgid (D2 #13). fflush() after.
362     * Then DO NOT wait: fall through, unblock SIGCHLD, return, so the
363     * shell prompts immediately (D2 #5). Completion gets reported
364     * later, asynchronously, by handle_dead() (D2 #10). */
365 }
366 for (int i = 0; i < stages; i++) if (opened[i]) redir_finish(&redirs[i]);
367 sigchld_unblock();
368 return;
369 }
370
371 /* ----- FOREGROUND ----- */
372 take_terminal(pgid); /* hand the tty to the job */
373 FgResult r = wait_group(pgid, npids); /* -> [TODO-2] */
374 take_terminal(shell_pgid); /* reclaim the tty */
375
376 for (int i = 0; i < stages; i++) if (opened[i]) redir_finish(&redirs[i]);
377
378 if (r == FG_STOPPED) {
379     /* Part E1 (later): a Ctrl-Z'd foreground job becomes a tracked
380     * STOPPED job here. For Part D you can leave this empty. */
381 }
382 sigchld_unblock();
383 }
384
385 /* ===== Foreground dispatch ===== */
386 #define RUN_OK 1
387 #define RUN_FAILED 0
388
389 /* A single builtin runs IN the shell process, so redirection is done by
390 * saving fd 0/1, dup2'ing, running, then restoring. */
391 static int run_builtin_with_redir(Command *cmd)
392 {
393     Redirection redir;
394     int saved_in = -1, saved_out = -1;
395
396     if (redir_open(cmd, &redir) == 0) return RUN_OK;
397     if (redir.in_fd >= 0) { saved_in = dup(STDIN_FILENO); dup2(redir.in_fd, STDIN_FILENO); }
398     if (redir.out_fd >= 0) { fflush(stdout); saved_out = dup(STDOUT_FILENO); dup2(redir.out_fd, STDOUT_FILENO); }
```



claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

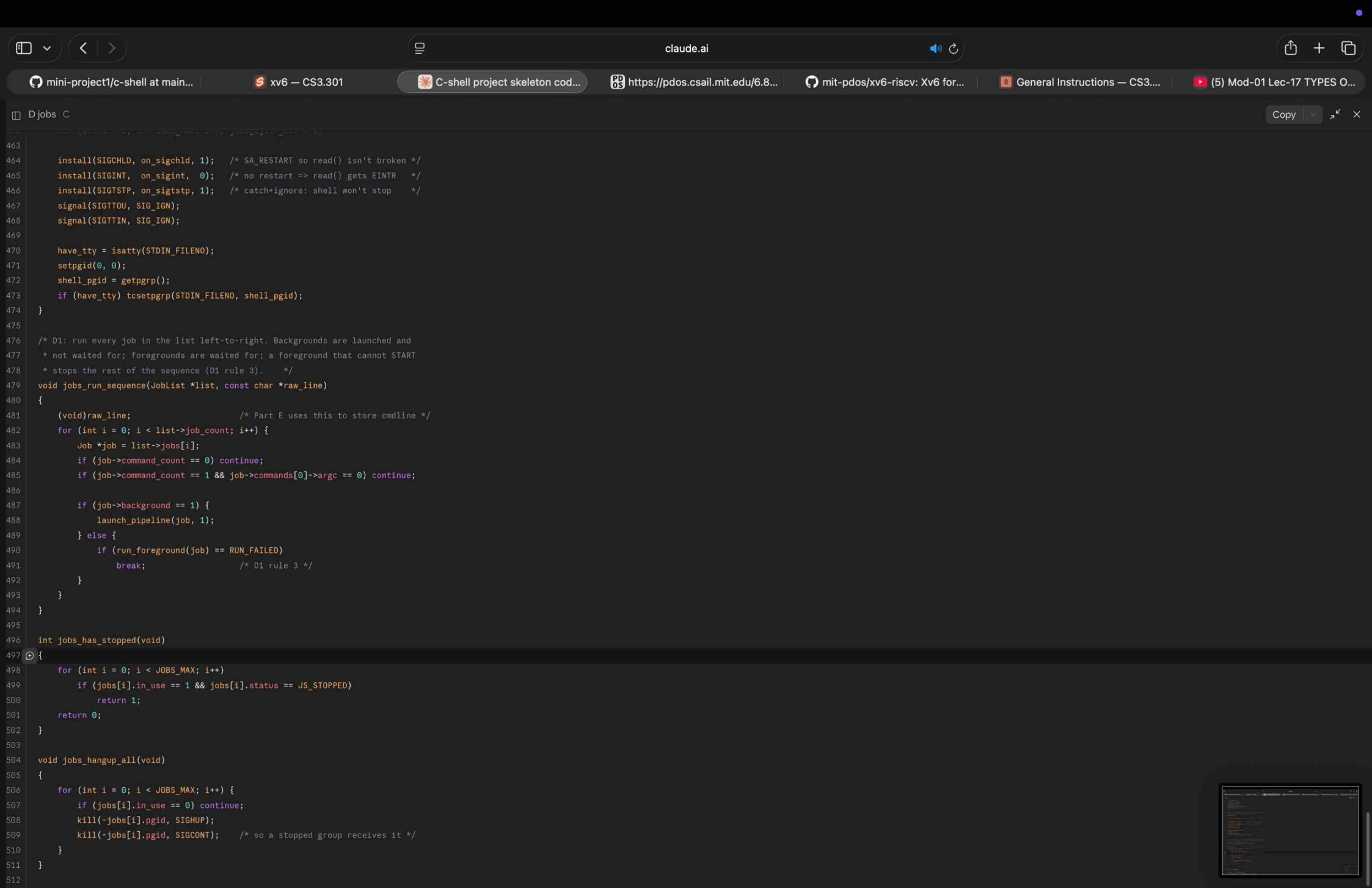
General Instructions — CS3...

(5) Mod-01 Lec-17 TYPES O...

D jobs C

Copy

```
450     struct sigaction sa;
451     memset(&sa, 0, sizeof(sa));
452     sa.sa_handler = handler;
453     sigemptyset(&sa.sa_mask);
454     sa.sa_flags = restart ? SA_RESTART : 0;
455     sigaction(signo, &sa, NULL);
456 }
457
458 /* Call once at startup (already wired into a_main.c). Installs the SIGCHLD
459  * reaper (D2 #6) and puts the shell in charge of the terminal.      */
460 void jobs_init(void)
461 {
462     for (int i = 0; i < JOBS_MAX; i++) jobs[i].in_use = 0;
463
464     install(SIGCHLD, on_sigchld, 1); /* SA_RESTART so read() isn't broken */
465     install(SIGINT, on_sigint, 0); /* no restart => read() gets EINTR */
466     install(SIGTSTP, on_sigstp, 1); /* catch+ignore: shell won't stop */
467     signal(SIGTTOU, SIG_IGN);
468     signal(SIGTTIN, SIG_IGN);
469
470     have_tty = isatty(STDIN_FILENO);
471     setpgid(0, 0);
472     shell_pgid = getpgrp();
473     if (have_tty) tcsetpgrp(STDIN_FILENO, shell_pgid);
474 }
475
476 /* D1: run every job in the list left-to-right. Backgrounds are launched and
477  * not waited for; foregrounds are waited for; a foreground that cannot START
478  * stops the rest of the sequence (D1 rule 3).      */
479 void jobs_run_sequence(JobList *list, const char *raw_line)
480 {
481     (void)raw_line; /* Part E uses this to store cmdline */
482     for (int i = 0; i < list->job_count; i++) {
483         Job *job = list->jobs[i];
484         if (job->command_count == 0) continue;
485         if (job->command_count == 1 && job->commands[0]->argc == 0) continue;
486
487         if (job->background == 1) {
488             launch_pipeline(job, 1);
489         } else {
490             if (run_foreground(job) == RUN_FAILED)
491                 break; /* D1 rule 3 */
492         }
493     }
494 }
495
496 int jobs_has_stopped(void)
497 {
498     for (int i = 0; i < JOBS_MAX; i++)
499         if (jobs[i].in_use == 1 && jobs[i].status == JS_STOPPED)
```



claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

C-shell project skeleton code

Share

Give the heavy skeleton for part E and then how to do the rest of it

Presented file

Part E lives in the same `d_jobs.c`. The signal handlers, `tcsetpgrp` handoff, Ctrl-D handling, and `SIGHUP` -on-exit are already in place, so Part E comes down to five `T000` s. In implementation order:

1. TODO-E2 — record a Ctrl-Z'd job. When a foreground pipeline stops, it isn't in the job table yet, so the `FG_STOPPED` branch allocates a slot and copies the pids in — you set its status to `JS_STOPPED` and print `[id] + Stopped <cmdline>`. This relies on `wait_group` returning `FG_STOPPED` on `WIFSTOPPED`, so make sure that path handles stops, not just exits. Test: run `sleep 100`, press Ctrl-Z → `[1] + Stopped sleep 100`, back at the prompt, shell never stops.

2. TODO-E1 — the `activities` process lines. The oldest-first grouping and group header are written; you write the inner line per process. Probe each pid with `kill(pid, 0)` and skip it if that fails, so exited members aren't listed; otherwise print two-space-indented `<pid> <name> <state>` with state from the job's `status`. Test: `sleep 100 &`, then `cat | sort &`, then `activities` — grouped output with `Running / Stopped`.

3. TODO-E3 — resume actions. Parsing and error strings are done; the job-control half is yours. Send `SIGCONT` to `-pgid` and mark Running; for `bg`, print `[id] + Running <cmdline>` and return without touching the terminal; for `fg`, echo the command line, then `take_terminal` → `wait_group(pgid, nalive, timeout)` → reclaim, and handle the result (`FG_DONE` free the slot, `FG_STOPPED` re-stop and print, `FG_TIMEDOUT` free and print `resume: job timed out`). Two gotchas: **re-find the job by id after waiting** — `SIGCHLD` may have freed the slot while you blocked — and the `--timeout` detection lives in `wait_group`'s `EINTR` branch (arming is already wired: when `g_alarm` is set mid-wait, send the group `SIGTERM`, reap, return `FG_TIMEDOUT`). Test: Ctrl-Z a `sleep 100`, then `resume %1 bg`, `resume %1 fg`, and `resume %1 fg --timeout 3` → `resume: job timed out`.

4. TODO-E4 — ping delivery. Signal validation and `% 64` are done; you resolve the target and send. `%n` → `job_find_by_id`, signal the whole group with `kill(-pgid, actual)`; a bare number → a pid, accepted only if `job_find_by_pid` shows this shell is tracking it, then `kill(pid, actual)`. Print `Sent signal <sig> to <target>` echoing the *original* number, not the modulo'd one; any unknown target or failed `kill` prints `ping: no such process found`. Ordering is already handled — signal validated before target lookup, and negatives fail `all_digits` so they're `invalid syntax` rather than reduced by the modulo. Test: `ping 4030 15`, `ping %2 9`, `ping 4030 79` (delivers 15, echoes 79), `ping 99999 9` → not found, `ping 4030 abc` and `ping 4030 -3` → invalid syntax. ↓

+ Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

D jobs C

Copy

```
1  /*
2  * d_jobs.c - Part D (sequential ';' and background '&' execution) and
3  *           Part E (activities, terminal control, resume, ping).
4  *
5  * Implemented here: the job table and its helpers, async-signal-safe output
6  * helpers, the SIGCHLD/SIGINT/SIGTSTP/SIGALRM setup, the pipeline
7  * fork/pipe/redirect wiring, the foreground dispatch, the D1 sequencer, the
8  * terminal give/reclaim helpers, and the argument parsing / job lookups and
9  * fixed output strings for activities / resume / ping.
10 *
11 * Marked TODO (see the inline notes) - the D and E logic to fill in:
12 *   D : completion reporting, foreground group-wait, process-group formation,
13 *       the "[job] pid" line, and the D1 not-found stop rule.
14 *   E1 : the per-process print inside activities.
15 *   E2 : recording a Ctrl-Z'd foreground pipeline as a stopped job.
16 *   E3 : SIGCONT + fg/bg terminal handling + the --timeout path in resume.
17 *   E4 : target resolution + signal delivery in ping.
18 *
19 * Compiles as-is under -std=c23 -Wall -Wextra -Werror; fill the TODOs to make
20 * it functional and drop the (void) placeholders as each is implemented.
21 */
22
23 #include "d_jobs.h"
24 #include "a_shell.h"
25 #include "b_builtins.h"
26 #include "c_exec.h"
27 #include "f_spy.h"
28 #include "f_snoop.h"
29
30 #include <errno.h>
31 #include <signal.h>
32 #include <stdio.h>
33 #include <stdlib.h>
34 #include <string.h>
35 #include <sys/wait.h>
36 #include <termios.h>
37 #include <unistd.h>
38
39 /* ---- Globals: the job table + a little shell state ----- */
40 static Job_t jobs[JOBS_MAX];
41 static int  next_job_id = 1; /* monotonic; never reused (D2 #4) */
42 static pid_t shell_pgid = 0; /* the shell's own process group */
43 static int  have_tty    = 0; /* is stdin a real terminal? */
44
45 static volatile sig_atomic_t g_alarm = 0; /* set by the SIGALRM handler (E3) */
46
47 /* Result of waiting on a foreground pipeline:
48 *   FG_DONE - every member terminated
49 *   FG_STOPPED - the pipeline was stopped with Ctrl-Z (E2)
50 *   FG_TIMEDOUT - a 'resume --timeout' deadline elapsed first (E3) */
51 enum fg_res { FG_DONE, FG_STOPPED, FG_TIMEDOUT };
```


claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

Djobs C

Copy

```
94  /* ===== Async-signal-safe printing =====
95  * printf() is NOT safe to call inside a signal handler. write() is. These
96  * two are what you use to print the completion message from handle_dead(). */
97  __attribute__((unused)) static void wstr(const char *s)
98  {
99      write(STDOUT_FILENO, s, strlen(s));
100  }
101
102  __attribute__((unused)) static void wuint(unsigned long v)
103  {
104      char buf[24];
105      int i = (int)sizeof(buf);
106      if (v == 0) { write(STDOUT_FILENO, "0", 1); return; }
107      while (v > 0 && i > 0) { buf[--i] = (char)('0' + (v % 10)); v /= 10; }
108      write(STDOUT_FILENO, buf + i, (size_t)((int)sizeof(buf) - i));
109  }
110
111  /* ---- Block SIGCHLD around code that touches the job table ----
112  * The SIGCHLD handler also touches the table, so we mask it while the main
113  * loop is mid-update to avoid a torn read/write. */
114  static void sigchld_block(void)
115  {
116      sigset_t s; sigemptyset(&s); sigaddset(&s, SIGCHLD);
117      sigprocmask(SIG_BLOCK, &s, NULL);
118  }
119  static void sigchld_unblock(void)
120  {
121      sigset_t s; sigemptyset(&s); sigaddset(&s, SIGCHLD);
122      sigprocmask(SIG_UNBLOCK, &s, NULL);
123  }
124
125  /* =====
126  * [TODO-1] handle_dead() - called from the SIGCHLD handler below
127  *
128  * Goal (D2 #8/#9/#10/#13): one pid of some pipeline just terminated.
129  * 1. Find its job with job_find_by_pid(pid). If NULL, return.
130  * 2. If it died from a signal ('signaled' != 0), mark j->abnormal = 1.
131  * 3. Decrement j->nalive (guard against going below 0).
132  * 4. When j->nalive == 0 the WHOLE pipeline is done, so print exactly:
133  *    <name> with pid <pgid> exited normally\n      (if !abnormal)
134  *    <name> with pid <pgid> exited abnormally\n      (if abnormal)
135  *    using wstr()/wuint() ONLY (async-safe). The reported pid is the
136  *    pipeline's pgid == first pid (D2 #13). Use j->name for <name>.
137  * 5. Free the slot: j->in_use = 0.
138  *
139  * Reminder: "exited normally" covers ANY WIFEXITED result - a non-zero
140  * exit status is still a normal exit (D2 #8).
141  * =====*/
142  static void handle_dead(pid_t pid, int signaled)
143  {
```

claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...Mini Project 1 — CS3.301(5) Mod-01 Lec-17 TYPES O...

Djobs C

Copy

```
144     Job_t *j = job_find_by_pid(pid);
145     if (j == NULL) return;
146
147     /* TODO-1: mark abnormal, decrement nalive, and when it hits 0 print the
148      *      "<name> with pid <pgid> exited normally/abnormally" line with
149      *      wstr()/wuint(), then set j->in_use = 0.          */
150     (void)signaled; /* remove once used */
151 }
152
153 /* ===== SIGCHLD handler =====
154  * D2 #6/#7: reap EVERY child that changed state, non-blocking, so the shell
155  * never blocks. WNOHANG => don't wait; loop until waitpid returns 0 / <0.
156  * WUNTRACED|WCONTINUED let us also notice bg jobs that stop/continue.      */
157 static void on_sigchld(int sig)
158 {
159     (void)sig;
160     int  saved = errno;      /* a handler must not clobber errno */
161     int  status;
162     pid_t pid;
163
164     while ((pid = waitpid(-1, &status, WNOHANG | WUNTRACED | WCONTINUED)) > 0) {
165         if (WIFEXITED(status) || WIFSIGNALED(status)) {
166             handle_dead(pid, WIFSIGNALED(status)); /* -> [TODO-1] */
167         } else if (WIFSTOPPED(status)) {
168             Job_t *j = job_find_by_pid(pid);
169             if (j) j->status = JS_STOPPED;
170         } else if (WIFCONTINUED(status)) {
171             Job_t *j = job_find_by_pid(pid);
172             if (j) j->status = JS_RUNNING;
173         }
174     }
175     errno = saved;
176 }
177
178 /* ---- Prompt-level signal handlers -----
179  * Ctrl-C: the shell itself must survive; drop the half-typed line.
180  * Ctrl-Z: the shell must NOT stop, so we catch-and-ignore at the prompt.      */
181 static void on_sigint(int sig)
182 {
183     (void)sig;
184     shell_sigint_flag = 1;
185     write(STDOUT_FILENO, "\n", 1);
186 }
187 static void on_sigtstp(int sig) { (void)sig; }
188
189 /* E3: the 'resume --timeout <s>' deadline. The handler only flips a flag; the
190  * wait loop checks it and does the actual SIGTERM + cleanup.          */
191 static void on_sigalrm(int sig) { (void)sig; g_alarm = 1; }
192
193 /* ===== Building display strings ===== */
194 static void display_job_status(Job_t *j, int max_colspan, char *buf, int len)
```

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

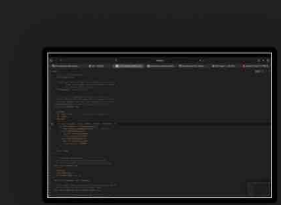
Mini Project 1 — CS3.301

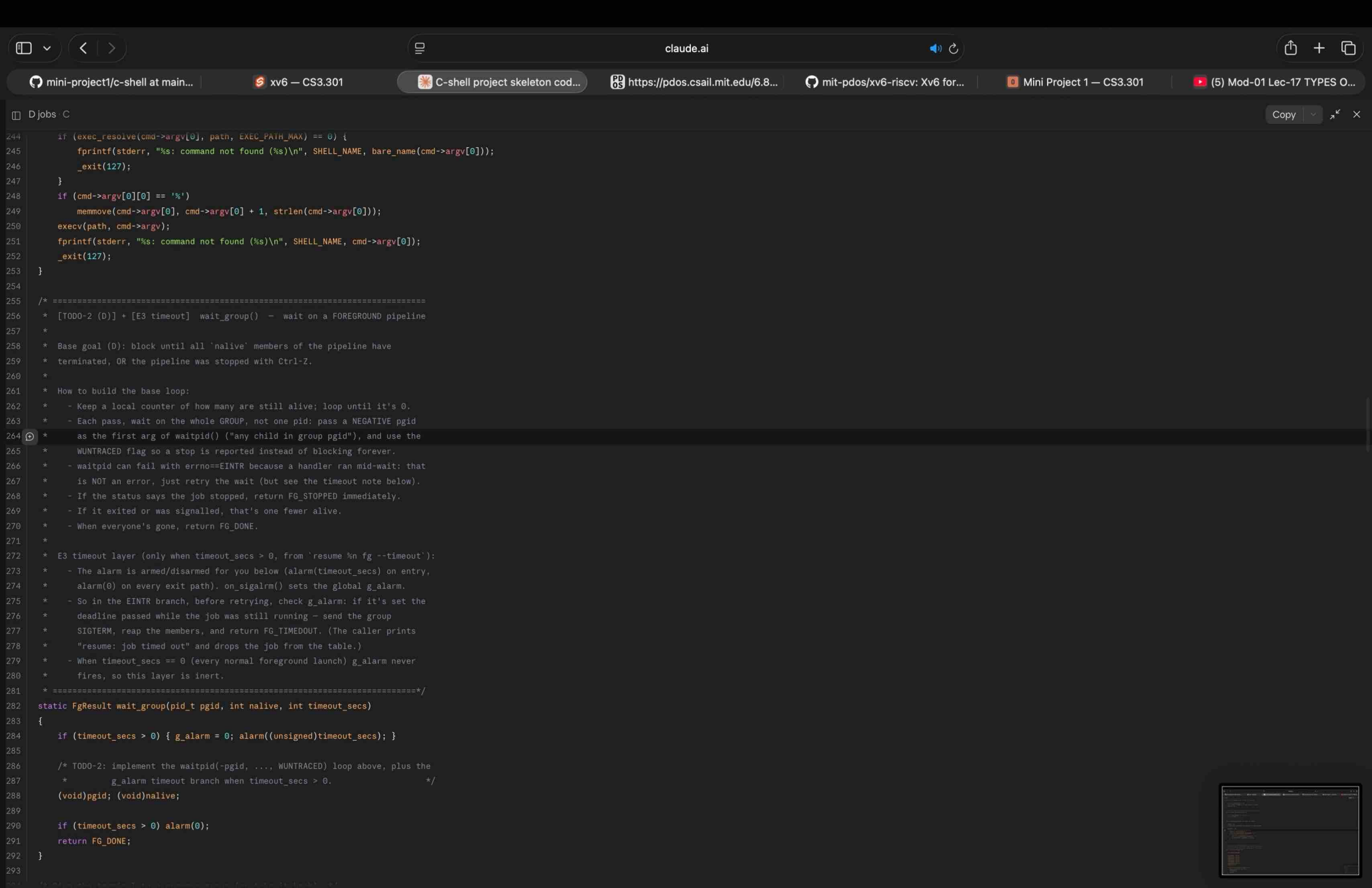
(5) Mod-01 Lec-17 TYPES O...

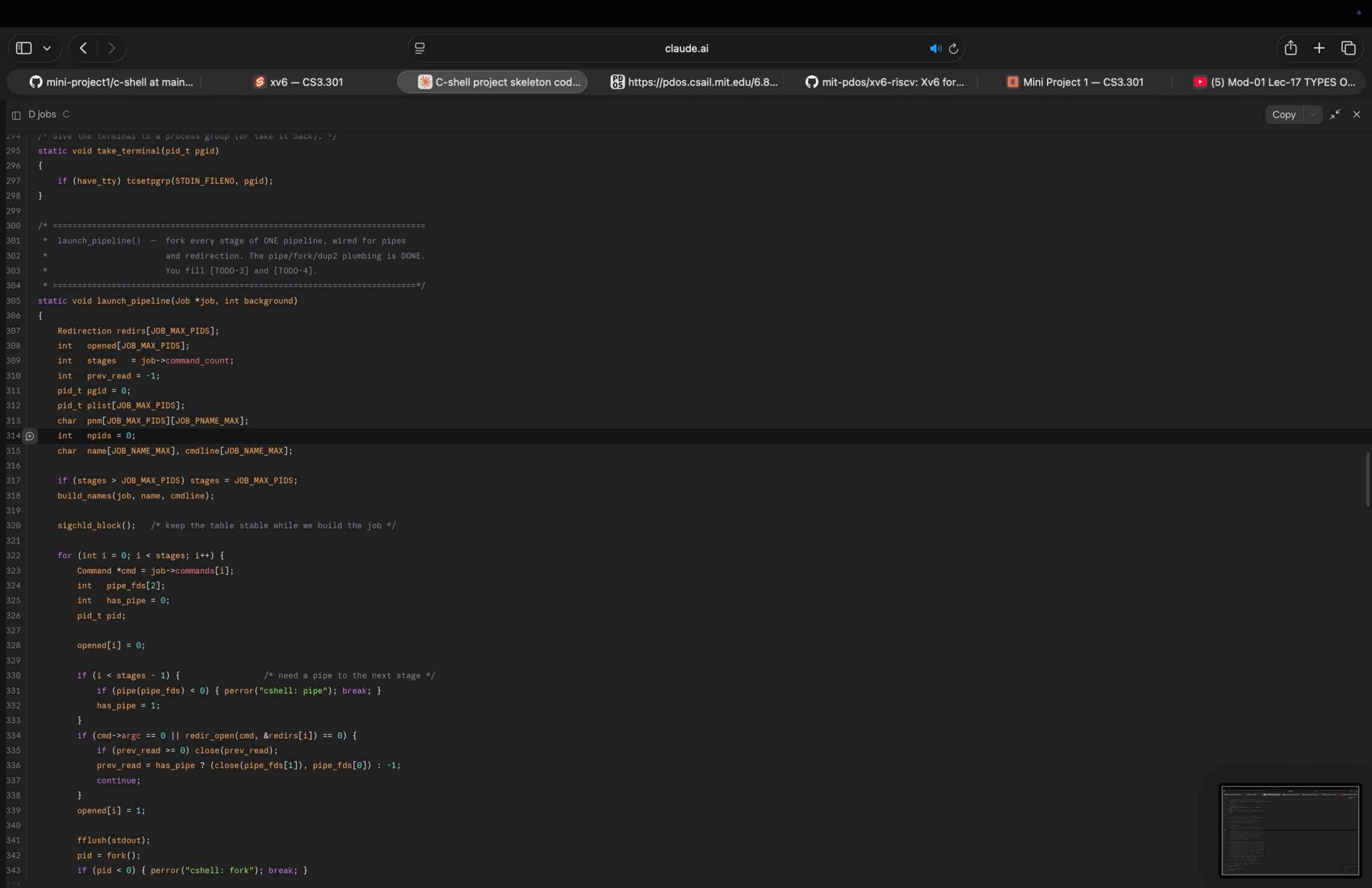
Djobs C

Copy

```
194 static void str_append(char *dst, int cap, const char *src)
195 {
196     int len = (int)strlen(dst), i = 0;
197     while (src[i] != '\0' && len + 1 < cap) dst[len++] = src[i++];
198     dst[len] = '\0';
199 }
200
201 /* "%ls" is our "skip cwd, go to PATH" syntax; the *name* is still "ls". */
202 static const char *bare_name(const char *s)
203 {
204     if (s != NULL && s[0] == '%') return s + 1;
205     return (s != NULL) ? s : "";
206 }
207
208 static void build_names(Job *job, char *name, char *cmdline)
209 {
210     name[0] = '\0';
211     str_append(name, JOB_NAME_MAX, bare_name(job->commands[0]->argv[0]));
212
213     cmdline[0] = '\0';
214     for (int s = 0; s < job->command_count; s++) {
215         Command *c = job->commands[s];
216         if (s > 0) str_append(cmdline, JOB_NAME_MAX, " | ");
217         for (int a = 0; a < c->argc; a++) {
218             if (a > 0) str_append(cmdline, JOB_NAME_MAX, " ");
219             str_append(cmdline, JOB_NAME_MAX, c->argv[a]);
220         }
221     }
222 }
223
224 /* ===== Child side after fork =====
225  * A forked child must (a) restore default signal behaviour so Ctrl-C/Ctrl-Z
226  * actually reach it (the shell has these handled/ignored), then (b) exec. */
227 static void child_run(Command *cmd)
228 {
229     char path[EXEC_PATH_MAX];
230
231     signal(SIGINT, SIG_DFL);
232     signal(SIGTSTP, SIG_DFL);
233     signal(SIGTTOU, SIG_DFL);
234     signal(SIGTTIN, SIG_DFL);
235     signal(SIGQUIT, SIG_DFL);
236     signal(SIGCHLD, SIG_DFL);
237     sigchld_unblock();
238
239     if (builtin_is_builtin(cmd->argv[0]) == 1) {
240         builtin_run(cmd->argv, cmd->argv);
241         fflush(stdout);
242         _exit(0);
243     }
```







⌕

⏪ ⏩

claude.ai

🔊 ↻

🔗 + 📄

🔍 mini-project1/c-shell at main...

🔍 xv6 — CS3.301

🔍 C-shell project skeleton cod...

🔍 https://pdos.csail.mit.edu/6.8...

🔍 mit-pdos/xv6-riscv: Xv6 for...

🔍 Mini Project 1 — CS3.301

🔍 (5) Mod-01 Lec-17 TYPES O...

D jobs C

Copy 🔍 ✕

```
344
345     if (pid == 0) {                /* ----- CHILD ----- */
346         /* [TODO-3a] Put this child in the pipeline's process group with
347          * setpgid(). Pass 0 as the pid arg (means "me") and the group
348          * leader's pgid as the group. For the FIRST stage pgid is still
349          * 0, which conveniently makes that child its own group leader -
350          * which is what you want (pgid == first pid, D2 #13).          */
351
352         if (prev_read >= 0) {        /* stdin from previous pipe */
353             dup2(prev_read, STDIN_FILENO);
354             close(prev_read);
355         }
356         if (has_pipe) {              /* stdout into next pipe */
357             close(pipe_fds[0]);
358             dup2(pipe_fds[1], STDOUT_FILENO);
359             close(pipe_fds[1]);
360         }
361         redir_apply(&redirs[i]);      /* < > >> override pipes */
362         for (int t = 0; t < redirs[i].target_count; t++)
363             close(redirs[i].targets[t]);
364         child_run(cmd);               /* never returns */
365     }
366
367     /* ----- PARENT ----- */
368     /* [TODO-3b] On the FIRST child, remember its pid as the group's pgid.
369      * Then call setpgid() from the PARENT too, to place this child in the
370      * group. Doing it on both sides is deliberate: after fork() you don't
371      * know whether parent or child runs first, and the child may even
372      * exec before you get here - whichever setpgid lands first wins, the
373      * other is a harmless no-op. Skip this and background jobs stay in the
374      * shell's group and fight for the terminal (breaks D2 #12).          */
375
376     plist[npids] = pid;
377     strncpy(pnm[npids], bare_name(cmd->argv[0]), JOB_PNAME_MAX - 1);
378     pnm[npids][JOB_PNAME_MAX - 1] = '\0';
379     npids++;
380
381     if (prev_read >= 0) close(prev_read);
382     prev_read = has_pipe ? (close(pipe_fds[1]), pipe_fds[0]) : -1;
383 }
384 if (prev_read >= 0) close(prev_read);
385
386 /* ----- BACKGROUND ----- */
387 if (background == 1) {
388     Job_t *slot = job_alloc();
389     if (slot != NULL) {
390         slot->pgid = pgid;
391         slot->npids = npids;
392         slot->nalive = npids;
393         slot->status = JS_RUNNING;
```

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

D jobs C

Copy

```
393     slot->status = JS_RUNNING;
394     memcpy(slot->pids, plist, sizeof(pid_t) * (size_t)npids);
395     for (int i = 0; i < npids; i++) strcpy(slot->pnames[i], pnm[i]);
396     str_append(slot->name, JOB_NAME_MAX, name);
397     str_append(slot->cmdline, JOB_NAME_MAX, cmdline);
398
399     /* [TODO-4] Print the background job line here, BEFORE returning and
400      * before any command output (D2 #3). Format is bracketed job id,
401      * a space, then the pid - match the spec example "[1] 4021" - and
402      * the pid to report is the group's pgid (D2 #13). fflush() after.
403      * Then DO NOT wait: fall through, unblock SIGCHLD, return, so the
404      * shell prompts immediately (D2 #5). Completion gets reported
405      * later, asynchronously, by handle_dead() (D2 #10). */
406 }
407 for (int i = 0; i < stages; i++) if (opened[i]) redir_finish(&redirs[i]);
408 sigchld_unblock();
409 return;
410 }
411
412 /* ----- FOREGROUND ----- */
413 take_terminal(pgid); /* E2 #2: hand the tty to the job */
414 FgResult r = wait_group(pgid, npids, 0); /* 0 = no timeout on a normal launch */
415 take_terminal(shell_pgid); /* E2 #3: reclaim the tty */
416
417 for (int i = 0; i < stages; i++) if (opened[i]) redir_finish(&redirs[i]);
418
419 if (r == FG_STOPPED) {
420     /* [E2 #5] Ctrl-Z stopped this foreground pipeline. Unlike a background
421      * launch it was never in the job table, so add it now: allocate a slot
422      * and copy the pids / pgid / names in (same fields the background branch
423      * above fills). The slot bookkeeping is below - you finish it: */
424     Job_t *slot = job_alloc();
425     if (slot != NULL) {
426         slot->pgid = pgid;
427         slot->npids = npids;
428         slot->nalive = npids;
429         memcpy(slot->pids, plist, sizeof(pid_t) * (size_t)npids);
430         for (int i = 0; i < npids; i++) strcpy(slot->pnames[i], pnm[i]);
431         str_append(slot->name, JOB_NAME_MAX, name);
432         str_append(slot->cmdline, JOB_NAME_MAX, cmdline);
433
434         /* [TODO-E2] Mark this slot's status as JS_STOPPED, then print the
435          * stopped notice in the spec's format - bracketed job id, " + ",
436          * "Stopped", then the command line, e.g. "[1] + Stopped sleep 100"
437          * (from slot->job_id and slot->cmdline). fflush() after. */
438     }
439 }
440 sigchld_unblock();
441 }
442
443 /* ----- FOREGROUND dispatch ----- */
```

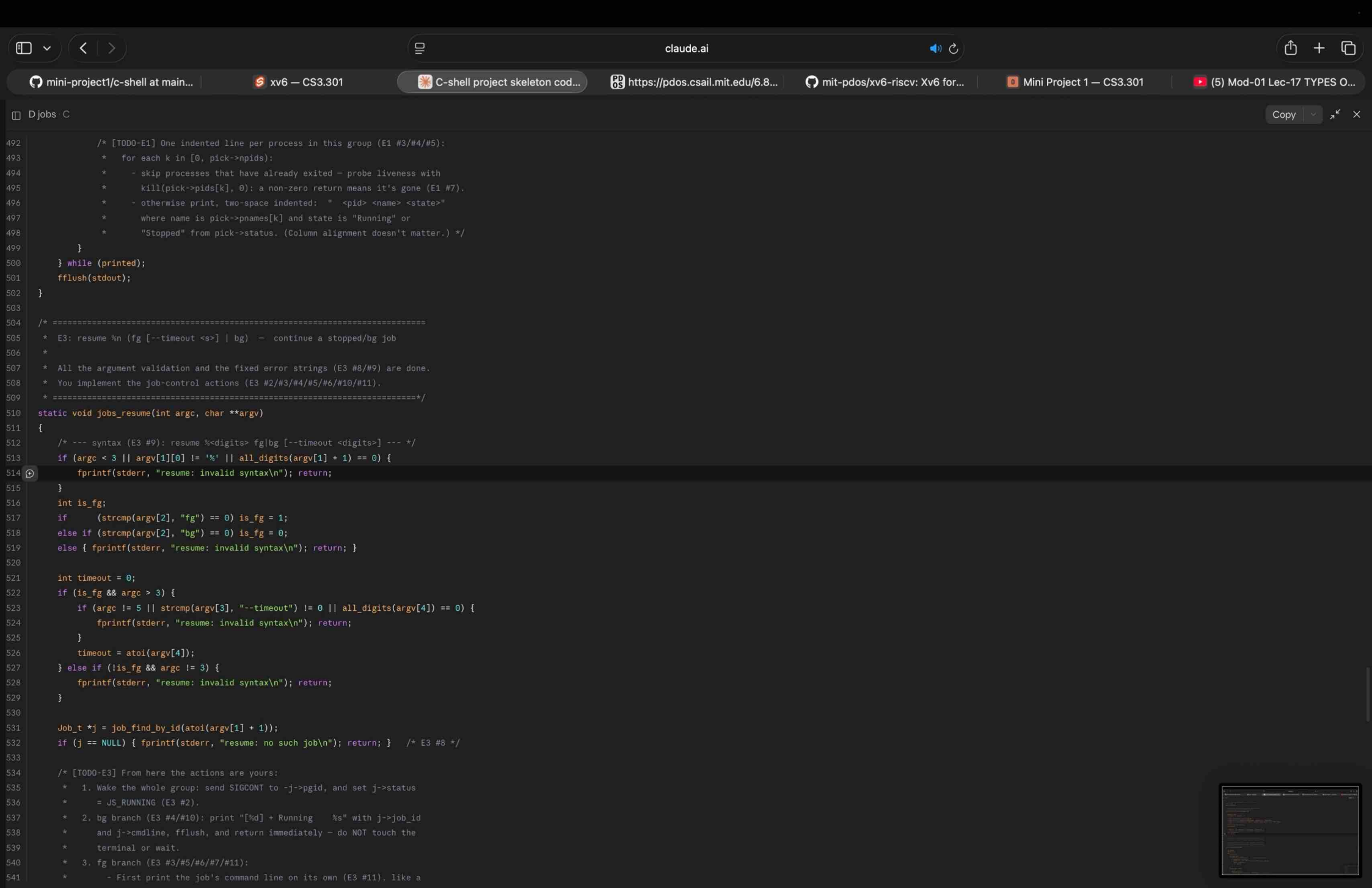
claude.ai

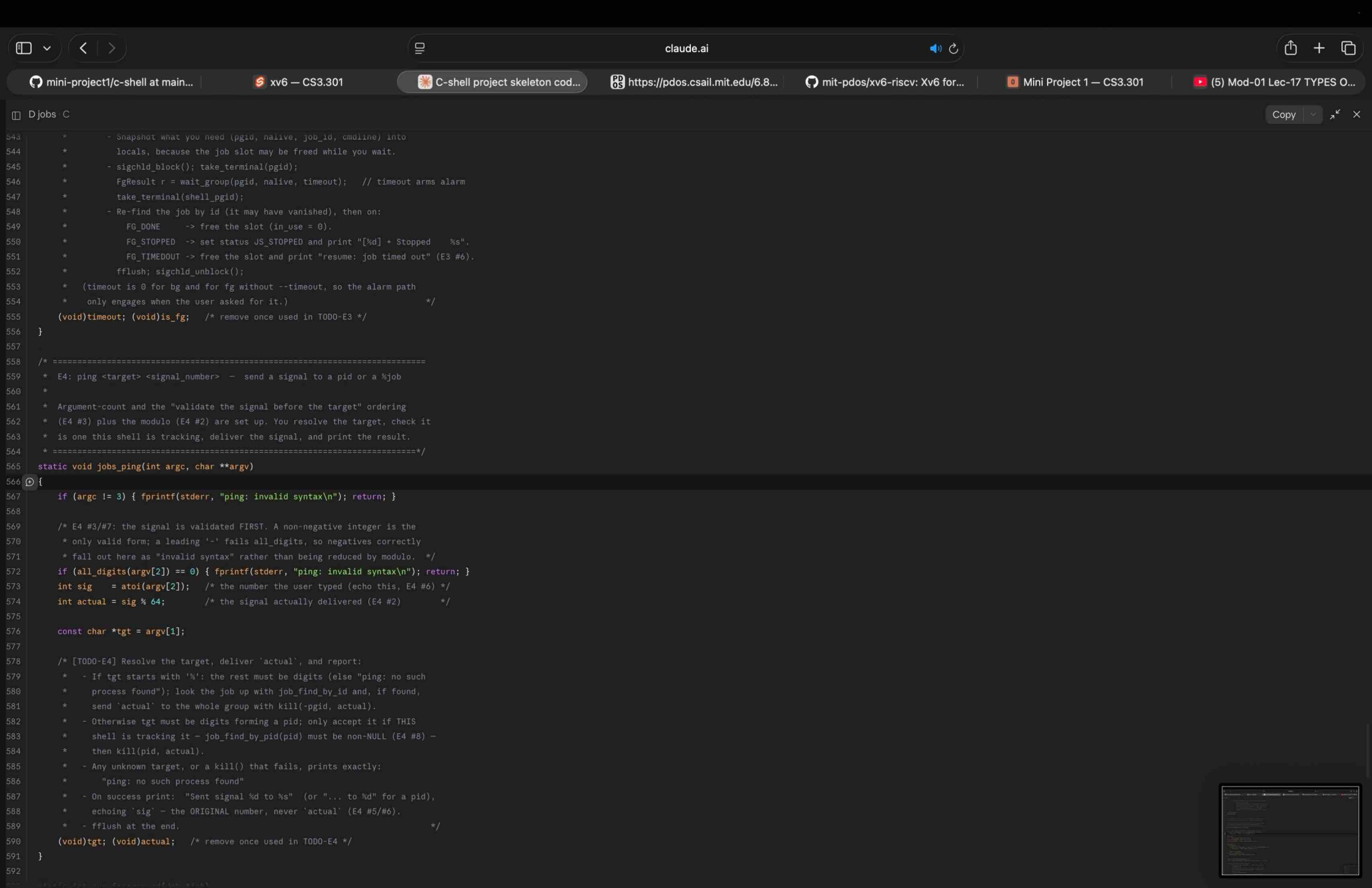
mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...Mini Project 1 — CS3.301(5) Mod-01 Lec-17 TYPES O...

D jobs · C

Copy

```
442
443 /* ===== Foreground dispatch ===== */
444 #define RUN_OK 1
445 #define RUN_FAILED 0
446
447 /* A single builtin runs IN the shell process, so redirection is done by
448  * saving fd 0/1, dup2'ing, running, then restoring. */
449 static int run_builtin_with_redir(Command *cmd)
450 {
451     Redirection redir;
452     int saved_in = -1, saved_out = -1;
453
454     if (redir_open(cmd, &redir) == 0) return RUN_OK;
455     if (redir.in_fd >= 0) { saved_in = dup(STDIN_FILENO); dup2(redir.in_fd, STDIN_FILENO); }
456     if (redir.out_fd >= 0) { fflush(stdout); saved_out = dup(STDOUT_FILENO); dup2(redir.out_fd, STDOUT_FILENO); }
457
458     builtin_run(cmd->argc, cmd->argv);
459     fflush(stdout);
460
461     if (saved_in >= 0) { dup2(saved_in, STDIN_FILENO); close(saved_in); }
462     if (saved_out >= 0) { dup2(saved_out, STDOUT_FILENO); close(saved_out); }
463     redir_finish(&redir);
464     return RUN_OK;
465 }
466
467 /* =====
468  * E1: activities - list tracked groups, oldest first, each with its members
469  *
470  * The outer selection walk below prints groups in ascending job_id (oldest
471  * first, E1 #6) and prints each group header. You fill the per-process line.
472  * =====*/
473 static void jobs_activities(void)
474 {
475     int printed;
476     int last_id = 0;
477     do {
478         int next_id = 0;
479         Job_t *pick = NULL;
480         for (int i = 0; i < JOBS_MAX; i++) { /* find the next job_id up */
481             if (jobs[i].in_use == 0) continue;
482             if (jobs[i].job_id > last_id && (next_id == 0 || jobs[i].job_id < next_id)) {
483                 next_id = jobs[i].job_id;
484                 pick = &jobs[i];
485             }
486         }
487         printed = (pick != NULL);
488         if (printed) {
489             last_id = pick->job_id;
490             printf("[%d] pgid %d\n", pick->job_id, (int)pick->pgid); /* E1 #5 group line */
491         }
492     } while (printed);
493 }
```





Copy

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

Djobs C

Copy

```
643     install(SIGCHLD, on_sigchld, 1); /* SA_RESTART so read() isn't broken */
644     install(SIGINT, on_sigint, 0); /* no restart => read() gets EINTR */
645     install(SIGTSTP, on_sigstsp, 1); /* E2 #1: catch+ignore, shell won't stop */
646     install(SIGALRM, on_sigalarm, 0); /* E3: no restart so wait_group sees EINTR */
647     signal(SIGTTOU, SIG_IGN); /* E2 #1: tcsetpgrp won't stop the shell */
648     signal(SIGTTIN, SIG_IGN);
649
650     have_tty = isatty(STDIN_FILENO);
651     setpgid(0, 0);
652     shell_pgid = getpgrp();
653     if (have_tty) tcsetpgrp(STDIN_FILENO, shell_pgid);
654 }
655
656 /* D1: run every job in the list left-to-right. Backgrounds are launched and
657  * not waited for; foregrounds are waited for; a foreground that cannot START
658  * stops the rest of the sequence (D1 rule 3). */
659 void jobs_run_sequence(JobList *list, const char *raw_line)
660 {
661     (void)raw_line; /* Part E uses this to store cmdline */
662     for (int i = 0; i < list->jobs_count; i++) {
663         Job *job = list->jobs[i];
664         if (job->command_count == 0) continue;
665         if (job->command_count == 1 && job->commands[0]->argc == 0) continue;
666
667         if (job->background == 1) {
668             launch_pipeline(job, 1);
669         } else {
670             if (run_foreground(job) == RUN_FAILED)
671                 break; /* D1 rule 3 */
672         }
673     }
674 }
675
676 int jobs_has_stopped(void)
677 {
678     for (int i = 0; i < JOBS_MAX; i++)
679         if (jobs[i].in_use == 1 && jobs[i].status == JS_STOPPED)
680             return 1;
681     return 0;
682 }
683
684 void jobs_hangup_all(void)
685 {
686     for (int i = 0; i < JOBS_MAX; i++) {
687         if (jobs[i].in_use == 0) continue;
688         kill(-jobs[i].pgid, SIGHUP);
689         kill(-jobs[i].pgid, SIGCONT); /* so a stopped group receives it */
690     }
691 }
692
```

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

C-shell project skeleton code

Share

Give the heavy skeleton for part F and tell how to do the rest

Ran a command

Part F is two files — `f_spy.c` (F1: read `/proc`) and `f_snoop.c` (F2: `ptrace` tracer). Let me build both skeletons.

Created 2 files, ran a command, shared files

How to complete each `TODO`:

F1 — `emit_fds()` in `f_spy.c` (one function, two gaps). `/proc/<pid>/fd` holds one entry per open descriptor; the entry name is the fd number and it's a symlink to whatever that fd points at. (a) Walk the directory with `readdir()`, keeping only entries whose name is numeric (that skips `.` and `..`), and `atoi()` each into `fds[n++]`. (b) After the provided sort, for each fd build `/proc/<pid>/fd/<n>`, `read_link()` it, and on success emit a row where the label is the fd number as text, the type comes from `type_of()` on the link path (stat follows it to the real object), and the path is the link target. `emit_link()` and `emit_maps()` right above it are the same pattern worked out in full — copy their shape. Test: `spy` with no argument lists the shell's own cwd, exe, mapped libraries, and fds (0/1/2 → your terminal); `spy <pid>` of a running process matches; a bad pid → `spy: no such process`.

F2 — the `ptrace` calls in `f_snoop.c` (four small gaps). The stepping loop, syscall table, timing, aggregation, sort, and print are all done; you supply the four `ptrace` calls that make tracing actually happen:

- F2b (child, command mode):** before `execv`, the child calls `ptrace(PTRACE_TRACEME, 0, 0, 0)`. That's what makes it stop on its `execve` and hand control to the parent — without it the parent's `waitpid` has nothing to catch.
- F2d (attach mode):** `ptrace(PTRACE_ATTACH, pid, 0, 0)`; if it fails, print `snoop: no such process` and return 1. Attaching injects a stop that the following `waitpid` collects.
- F2c (both modes):** after that first stop, `ptrace(PTRACE_SETOPTIONS, pid, 0, PTRACE_O_TRACESYSGOOD)`. This is what makes syscall stops arrive as `SIGTRAP | 0x80`, which is exactly the bit `run_loop` tests to tell a syscall-stop from an ordinary signal.
- F2a (inside `run_loop`, syscall entry):** declare a `struct user_regs_struct regs`, call `ptrace(PTRACE_GETREGS, pid, 0, ®s)`, and set `cur = (long)regs.orig_rax`. Use `orig_rax`, not `rax` — the kernel overwrites `rax` with the return value, but `orig_rax` preserves the syscall number across the call.

Test: `snoop ls` prints a table of syscalls with counts and time (expect `execve`, `openat`, `read`, `write`, `close`, `mmap`, `exit_group`); `snoop -p ↓ d>` of a shell-spawned process attaches and traces until it exits; a bad command → `snoop: command not found`, a bad pid → `snoop:`

+ Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

F spy C

Copy

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

/*

* f_spy.c - F1: spy [pid]

*

* Lists the open files of a process (or the shell itself, when no pid is given)

* by reading /proc/<pid>/. Prints a table: PID, FD, TYPE, PATH - covering the

* cwd, the executable (txt), memory-mapped files, and the numbered descriptors.

*

* Implemented here: the stat->TYPE mapping, the readlink/print helpers, the cwd

* and exe symlink rows, the /proc/<pid>/maps reader, and spy_run's argument

* parsing / existence + permission checks / table header.

*

* Marked TODO: emit_fds() - reading /proc/<pid>/fd. Use emit_link() and

* emit_maps() below as worked examples of the same /proc pattern.

*

* Compiles as-is under -std=c23 -Wall -Wextra -Werror.

*/

#include "f_spy.h"

#include "a_shell.h"

#include <dirent.h>

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <sys/stat.h>

#include <unistd.h>

#include <errno.h>

/* Map a path's stat mode to the short TYPE label spy prints. */

static const char *type_of(const char *path)

{

struct stat s;

if (stat(path, &s) != 0) return "unknown";

if (S_ISREG(s.st_mode)) return "REG";

if (S_ISDIR(s.st_mode)) return "DIR";

if (S_ISCHR(s.st_mode)) return "CHR";

if (S_ISBLK(s.st_mode)) return "BLK";

if (S_ISFIFO(s.st_mode)) return "FIFO";

if (S_ISSOCK(s.st_mode)) return "unix";

if (S_ISLNK(s.st_mode)) return "LINK";

return "unknown";

}

/* readlink() wrapper that NUL-terminates. Returns 1 on success, 0 on failure. */

static int read_link(const char *p, char *buf, size_t n)

{

ssize_t r = readlink(p, buf, n - 1);

if (r < 0) return 0;

buf[r] = '\0';

return 1;

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

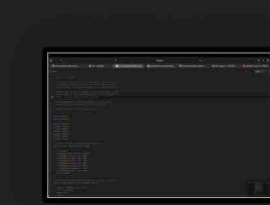
Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

F spy C

Copy

```
52
53 /* One table row. */
54 static void row(pid_t pid, const char *fd, const char *type, const char *path)
55 {
56     printf("%-6d %-6s %-6s %s\n", (int)pid, fd, type, path);
57 }
58
59 /* A single symlink under /proc/<pid>/ (used for cwd and exe). Worked example of
60  * the pattern emit_fds() needs: build the /proc path, readlink it, print a row. */
61 static void emit_link(pid_t pid, const char *label, const char *rel)
62 {
63     char link[128], tgt[4096];
64     snprintf(link, sizeof link, "/proc/%d/%s", (int)pid, rel);
65     if (read_link(link, tgt, sizeof tgt))
66         row(pid, label, type_of(tgt), tgt);
67 }
68
69 /* Each unique memory-mapped file path from /proc/<pid>/maps, printed once.
70  * Worked example of reading and parsing a /proc text file. */
71 static void emit_maps(pid_t pid)
72 {
73     char path[128];
74     snprintf(path, sizeof path, "/proc/%d/maps", (int)pid);
75     FILE *f = fopen(path, "r");
76     if (f == NULL) return;
77
78     char seen[512][4096];
79     int nseen = 0;
80     char line[8192];
81
82     while (fgets(line, sizeof line, f) != NULL) {
83         char *p = strchr(line, '/'); /* pathname starts at first '/' */
84         if (p == NULL) continue; /* anon / [heap] / [stack] */
85         size_t len = strlen(p);
86         if (len > 0 && p[len - 1] == '\n') p[--len] = '\0';
87
88         int dup = 0;
89         for (int i = 0; i < nseen; i++)
90             if (strcmp(seen[i], p) == 0) { dup = 1; break; }
91         if (dup) continue;
92
93         if (nseen < 512) {
94             strncpy(seen[nseen], p, 4095);
95             seen[nseen][4095] = '\0';
96             nseen++;
97         }
98         row(pid, "mem", type_of(p), p);
99     }
100     fclose(f);
101 }
```



claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

F spy C

Copy

```
102
103 static int cmp_int(const void *a, const void *b)
104 {
105     int x = *(const int *)a, y = *(const int *)b;
106     return (x > y) - (x < y);
107 }
108
109 /* =====
110 * [TODO-F1] emit_fds() - the numbered open descriptors, ascending
111 *
112 * The directory /proc/<pid>/fd contains one entry per open descriptor; the
113 * entry name is the fd number, and it is a symlink to whatever that fd refers
114 * to (a file, pipe, socket, tty, ...).
115 *
116 * Two gaps to fill (the opendir/closedir + qsort scaffolding is provided):
117 * (a) Read the directory with readdir(). Keep only entries whose name is a
118 *     number (skip "." / ".."), converting each with atoi() into fds[n++].
119 * (b) After the sort, for each fd: build "/proc/<pid>/fd/<n>", read_link()
120 *     it into a buffer, and if that succeeds emit a row: the label is the fd
121 *     number as text, the type comes from type_of() on the LINK path itself
122 *     (stat follows it to the real object), and the path is the link target.
123 *
124 * Model both gaps on emit_link()/emit_maps() above - same helpers, same shape.
125 * =====*/
126 static void emit_fds(pid_t pid)
127 {
128     char dirp[128];
129     snprintf(dirp, sizeof dirp, "/proc/%d/fd", (int)pid);
130     DIR *d = opendir(dirp);
131     if (d == NULL) return;
132
133     int fds[1024];
134     int n = 0;
135
136     /* TODO-F1(a): readdir(d) and collect numeric entry names into fds[0..n). */
137
138     closedir(d);
139     qsort(fds, n, sizeof(int), cmp_int);
140
141     /* TODO-F1(b): for each fds[i], read the /proc/<pid>/fd/<i> symlink and,
142      * on success, row(pid, "<fd number>", type_of(link), target). */
143 }
144
145 int spy_run(int argc, char **argv)
146 {
147     pid_t pid;
148
149     if (argc == 1) pid = getpid(); /* no arg -> the shell itself */
150     else if (argc == 2) pid = (pid_t)atoi(argv[1]);
151     else { fprintf(stderr, "spy: invalid syntax\n"); return 1; }
```

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

F spy C

Copy

```
123 *
124 * Model both gaps on emit_link()/emit_maps() above - same helpers, same shape.
125 * =====*/
126 static void emit_fds(pid_t pid)
127 {
128     char dirp[128];
129     snprintf(dirp, sizeof dirp, "/proc/%d/fd", (int)pid);
130     DIR *d = opendir(dirp);
131     if (d == NULL) return;
132
133     int fds[1024];
134     int n = 0;
135
136     /* TODO-F1(a): readdir(d) and collect numeric entry names into fds[0..n). */
137
138     closedir(d);
139     qsort(fds, n, sizeof(int), cmp_int);
140
141     /* TODO-F1(b): for each fds[i], read the /proc/<pid>/fd/<i> symlink and,
142      *      on success, row(pid, "<fd number>", type_of(link), target). */
143 }
144
145 int spy_run(int argc, char **argv)
146 {
147     pid_t pid;
148
149     if (argc == 1) pid = getpid();          /* no arg -> the shell itself */
150     else if (argc == 2) pid = (pid_t)atoi(argv[1]);
151     else { fprintf(stderr, "spy: invalid syntax\n"); return 1; }
152
153     char base[64];
154     struct stat st;
155     snprintf(base, sizeof base, "/proc/%d", (int)pid);
156     if (stat(base, &st) != 0) { fprintf(stderr, "spy: no such process\n"); return 1; }
157
158     char fddir[80];
159     snprintf(fddir, sizeof fddir, "/proc/%d/fd", (int)pid);
160     DIR *probe = opendir(fddir);
161     if (probe == NULL && errno == EACCES) { fprintf(stderr, "spy: permission denied\n"); return 1; }
162     if (probe != NULL) closedir(probe);
163
164     printf("%-6s %-6s %-6s %s\n", "PID", "FD", "TYPE", "PATH");
165     emit_link(pid, "cwd", "cwd");
166     emit_link(pid, "txt", "exe");
167     emit_maps(pid);
168     emit_fds(pid);
169     fflush(stdout);
170     return 0;
171 }
172
```

claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

F snoop C

Copy

1

/*

2

* f_snoop.c - F2: snoop <cmd [args...]> | snoop -p <pid>

3

*

4

* Traces a process with ptrace, counts every system call it makes (with total

5

* time spent in each), and prints a summary when the tracee exits. Two modes:

6

* launch a command and trace it from the first execve, or attach to a running

7

* pid the shell already knows about.

8

*

9

* Implemented here: the SIGCHLD guard (so the shell's own reaper doesn't steal

10

* the tracee's stops), the x86-64 syscall-number->name table and lookup, the

11

* per-syscall aggregation + count-desc/first-seen sort, the summary print, and

12

* snoop_run's argument parsing / command resolution / fork.

13

*

14

* Marked TODO - the ptrace mechanics:

15

* F2a : read the syscall number on each entry stop (PTRACE_GETREGS).

16

* F2b : the traced child announces itself (PTRACE_TRACEME) before execve.

17

* F2c : after the first stop, ask for syscall-good stops (PTRACE_SETOPTIONS).

18

* F2d : attach mode grabs the target (PTRACE_ATTACH).

19

*

20

* Compiles as-is under -std=c23 -Wall -Wextra -Werror.

21

*/

22

23

#include "f_snoop.h"

24

#include "a_shell.h"

25

#include "c_exec.h"

26

27

#include <errno.h>

28

#include <signal.h>

29

#include <stdio.h>

30

#include <stdlib.h>

31

#include <string.h>

32

#include <sys/ptrace.h>

33

#include <sys/stat.h>

34

#include <sys/types.h>

35

#include <sys/user.h>

36

#include <sys/wait.h>

37

#include <time.h>

38

#include <unistd.h>

39

40

#define NSYS 512

41

42

/* The shell's SIGCHLD handler would reap our tracee's stops; block SIGCHLD for

43

* the duration of tracing so ptrace sees every wait status. */

44

static void snoop_sigchld(int block)

45

{

46

sigset_t s;

47

sigemptyset(&s);

48

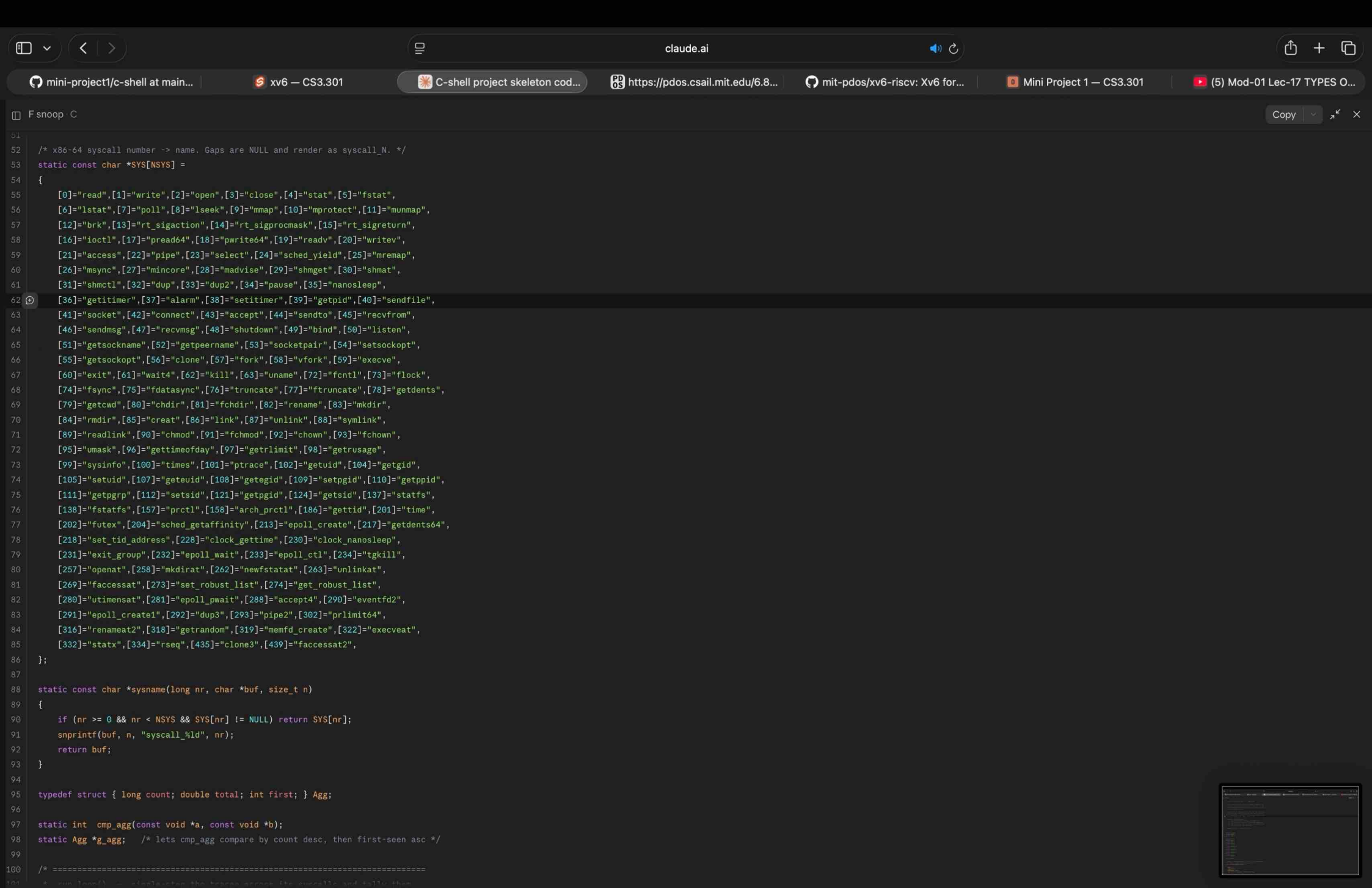
sigaddset(&s, SIGCHLD);

49

sigprocmask(block ? SIG_BLOCK : SIG_UNBLOCK, &s, NULL);

50

}



claude.ai

mini-project1/c-shell at main...

xv6 — CS3.301

C-shell project skeleton cod...

https://pdos.csail.mit.edu/6.8...

mit-pdos/xv6-riscv: Xv6 for...

Mini Project 1 — CS3.301

(5) Mod-01 Lec-17 TYPES O...

F snoop C

Copy

```
149     }
150     }
151 }
152
153 long used[NSYS];
154 int nused = 0;
155 for (int i = 0; i < norder; i++) used[nused++] = order_nr[i];
156 g_agg = agg;
157 qsort(used, nused, sizeof(long), cmp_agg);
158
159 printf("%-18s%-8s\n", "syscall", "calls", "time");
160 for (int i = 0; i < nused; i++) {
161     char buf[32];
162     long nr = used[i];
163     printf("%-18s%-8ld%.3fs\n", sysname(nr, buf, sizeof buf), agg[nr].count, agg[nr].total);
164 }
165 fflush(stdout);
166 }
167
168 static int cmp_agg(const void *a, const void *b)
169 {
170     long x = *(const long *)a, y = *(const long *)b;
171     if (g_agg[x].count != g_agg[y].count)
172         return (g_agg[x].count < g_agg[y].count) - (g_agg[x].count > g_agg[y].count);
173     return (g_agg[x].first > g_agg[y].first) - (g_agg[x].first < g_agg[y].first);
174 }
175
176 int snoop_run(int argc, char **argv)
177 {
178     /* ---- attach mode: snoop -p <pid> ---- */
179     if (argc >= 3 && strcmp(argv[1], "-p") == 0) {
180         pid_t pid = (pid_t)atoi(argv[2]);
181         char proc[64];
182         struct stat st;
183         snprintf(proc, sizeof proc, "/proc/%d", (int)pid);
184         if (stat(proc, &st) != 0) { fprintf(stderr, "snoop: no such process\n"); return 1; }
185
186         /* TODO-F2d: attach to the target with ptrace(PTRACE_ATTACH, pid, 0, 0).
187          * If it fails, print "snoop: no such process" and return 1. Attaching
188          * sends the tracee a stop that the waitpid() below then collects. */
189
190         int status;
191         snoop_sigchld(1);
192         waitpid(pid, &status, 0); /* the attach stop */
193         /* TODO-F2c: ptrace(PTRACE_SETOPTIONS, pid, 0, PTRACE_O_TRACESYSGOOD)
194          * so syscall stops arrive as (SIGTRAP|0x80), which run_loop checks. */
195         run_loop(pid);
196         snoop_sigchld(0);
197         return 0;
198     }
199 }
```

claude.ai

mini-project1/c-shell at main...xv6 — CS3.301C-shell project skeleton cod...https://pdos.csail.mit.edu/6.8...mit-pdos/xv6-riscv: Xv6 for...Mini Project 1 — CS3.301(5) Mod-01 Lec-17 TYPES O...

F snoop.cCopy

```
182     struct stat st;
183     snprintf(proc, sizeof proc, "/proc/%d", (int)pid);
184     if (stat(proc, &st) != 0) { fprintf(stderr, "snoop: no such process\n"); return 1; }
185
186     /* TODO-F2d: attach to the target with ptrace(PTRACE_ATTACH, pid, 0, 0).
187      * If it fails, print "snoop: no such process" and return 1. Attaching
188      * sends the tracee a stop that the waitpid() below then collects. */
189
190     int status;
191     snoop_sigchld(1);
192     waitpid(pid, &status, 0); /* the attach stop */
193     /* TODO-F2c: ptrace(PTRACE_SETOPTIONS, pid, 0, PTRACE_O_TRACESYSGOOD)
194      * so syscall stops arrive as (SIGTRAP|0x80), which run_loop checks. */
195     run_loop(pid);
196     snoop_sigchld(0);
197     return 0;
198 }
199
200 /* ---- command mode: snoop <cmd> [args...] ---- */
201 if (argc >= 2) {
202     char path[EXEC_PATH_MAX];
203     if (exec_resolve(argv[1], path, EXEC_PATH_MAX) == 0) {
204         fprintf(stderr, "snoop: command not found\n");
205         return 1;
206     }
207     pid_t pid = fork();
208     if (pid < 0) { perror("snoop: fork"); return 1; }
209
210     if (pid == 0) { /* ---- tracee ---- */
211         snoop_sigchld(0); /* clean mask for the child */
212         /* TODO-F2b: ptrace(PTRACE_TRACEME, 0, 0, 0) so this child becomes
213          * traceable; it will then stop on the execve below and hand
214          * control back to the parent. */
215         execv(path, &argv[1]);
216         _exit(127);
217     }
218
219     int status;
220     snoop_sigchld(1);
221     waitpid(pid, &status, 0); /* stop after execve */
222     /* TODO-F2c: PTRACE_SETOPTIONS with PTRACE_O_TRACESYSGOOD (as above). */
223     run_loop(pid);
224     snoop_sigchld(0);
225     return 0;
226 }
227
228 fprintf(stderr, "snoop: invalid syntax\n");
229 return 1;
230 }
231
```

Working through an xv6 operating systems assignment.

[Fetches: Mini Project 1 — CS3.301](#) >

Let me first set up the environment and check what tooling is available.

Ran 2 commands >

Ran 5 commands >

Ran a command >

Ran a command >

+ Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 Medium

Copy

1	[MLFQ]	t=0	pid=1	q=0	ev=NEW
2	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
3	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
4	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
5	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
6	Ⓢ [MLFQ]	t=1	pid=1	q=0	ev=SLEEP
7	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
8	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
9	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
10	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
11	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
12	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
13	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
14	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
15	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
16	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
17	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
18	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
19	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
20	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
21	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
22	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
23	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
24	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
25	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
26	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
27	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
28	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
29	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
30	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
31	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
32	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
33	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
34	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
35	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
36	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
37	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
38	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
39	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
40	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
41	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
42	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP

Ran 9 commands >

- Timer ticks call `yield()` **unconditionally** every tick in both `usertrap` and `kerneltrap` (`trap.c`). This is RR. MLFQ must replace this with tick-accounting logic.
- `allocproc` is the single choke point for process creation; `kfork` sets the new proc **RUNNABLE** at the end.
- Voluntary yield happens via `sleep()` (RUNNING→SLEEPING); becoming runnable again happens in `wakeup()` (SLEEPING→RUNNABLE). These are the hooks for Rule 5.
- `kexit` → ZOMBIE. The scheduler only ever runs **RUNNABLE** procs.
- Global `ticks` lives in `trap.c`; `clockintr` bumps it on `cpu0` 0 — the natural place for the 48-tick boost.

Ran a command >

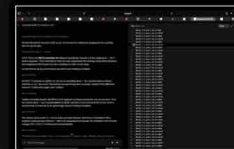
I now have everything I need. Let me lay out the plan before touching code so you can follow the theory:

1. **Represent the 4 queues** by a `priority` field (0-3) on each process, with FIFO order *within* a queue tracked by a monotonic `enter_seq` counter. This is probably equivalent to explicit array-queues but avoids xv6's nastiest deadlock traps (holding `p->lock` while touching a shared queue in `sleep / wakeup`). I'll document this choice for the viva.
2. **Selection** (Rule 2, strict priority + Rule 6, RR in Q3): scheduler scans for the RUNNABLE proc with the lowest `priority`, breaking ties by lowest `enter_seq` (= oldest in queue = FIFO/round-robin).
3. **Time slices** `{1,4,8,16}` counted per tick (Rule 2.1.2).
4. **Slice exhaustion** (Rule 4): on tick, if `ticks_used` hits the slice, demote to next lower queue (or stay at Q3), re-inserted at tail (new `enter_seq`).
5. **Preemption** (Rule 2): on tick, if a higher-priority queue is non-empty, yield without demoting.
6. **Voluntary yield** (Rule 5): `sleep()` resets `ticks_used` (original gameable MLFQ rule — key for the report's I/O-bound analysis); `wakeup()` re-inserts at tail of the *same* queue, priority unchanged.
7. **Boost** (Rule 7): every 48 ticks, in `clockintr`, `~"` processes $\rightarrow$ Q0.
8. `#ifndef MLFQ` guards everywhere so the RR `↓` `completes` unchanged when `SCHEDULER` is

Opus 4.8 Medium

Copy

1	[MLFQ]	t=0	pid=1	q=0	ev=NEW
2	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
3	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
4	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
5	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
6	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
7	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
8	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
9	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
10	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
11	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
12	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
13	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
14	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
15	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
16	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
17	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
18	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
19	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
20	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
21	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
22	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
23	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
24	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
25	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
26	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
27	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
28	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
29	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
30	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
31	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
32	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
33	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
34	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
35	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
36	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
37	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
38	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
39	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
40	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
41	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
42	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
43	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
44	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
45	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
46	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
47	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
48	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
49	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
50	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP



- Implementing MLFQ scheduler in xv6

6. **Voluntary yield** (Rule 5): `sleep()` resets `ticks_used` (original gameable MLFQ rule — key for the report's I/O-bound analysis); `wakeup()` re-inserts at tail of the *same* queue, priority unchanged.
7. **Boost** (Rule 7): every 48 ticks, in `clockintr`, all processes → Q0.
8. **`#ifdef MLFQ` guards** everywhere so the RR path compiles unchanged when `SCHEDULER` is unset.

Let me verify the include chains before editing (so my new symbols resolve).

Edited 6 files, ran 14 commands >

Status: core MLFQ implemented and verified

I have the RISC-V toolchain + QEMU installed and a working **build** → **boot** → **run** → **verify** loop. The core MLFQ scheduler (Section 2.1, the [70]-mark bulk) is written and passes every check I can run so far.

What's done and verified:

- **Both configs build clean** with `-Wall -Werror: make clean; make qemu` (RR, unchanged path) and `make clean; make qemu SCHEDULER=MLFQ`. Confirmed `-D MLFQ` actually reaches the compiler.
- **MLFQ kernel boots and runs userspace** (`echo`, `ls`, fork/exec, console + disk I/O) with no panic on `CPUS=1`.
- **`procdump` (Ctrl+P) works** and prints the new columns: PID, state, name, queue, ticks used in slice, enqueue-seq, creation tick.

What each change implements (all `#ifdef MLFQ`-guarded so RR is untouched):

- **Makefile** — `SCHEDULER=MLFQ` macro; `unset → RR`.
- **struct proc** — `priority`, `ticks_used`, `enter_seq` + metric fields (`ctime/stime/etime/rtime`).
- **allocproc** — Rule 1: new proc → tail of Q0.
- **scheduler** — Rule 2/6: strict priority, FIFO within a queue via `enter_seq` (round-robin in Q3 falls out naturally). Deadlock-free ascending lock order.
- **mlfq_tick_yield** (trapc timer hook) — slice accounting, Rule 4 demotion, Rule 2 preemption.
- **sleep / wakeup** — Rule 5: voluntary yield resets slice, re-inserts at tail of same queue.
- **clockintr** — Rule 7: boost all procs to Q0 every 48 ticks.

Files + a reviewable patch are saved:

- `mlfq_changes.patch` (215 insertions, apply with `git apply`)
- Modified `Makefile`, `kernel/{param,proc,defs}.h`, `kernel/proc.c`

+ Write a message...

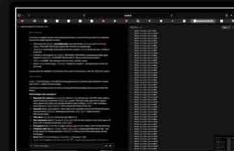
Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 Medium

Copy

Copy

1	[MLFQ]	t=0	pid=1	q=0	ev=NEW
2	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
3	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
4	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
5	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
6	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
7	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
8	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
9	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
10	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
11	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
12	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
13	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
14	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
15	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
16	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
17	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
18	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
19	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
20	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
21	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
22	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
23	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
24	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
25	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
26	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
27	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
28	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
29	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
30	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
31	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
32	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
33	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
34	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
35	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
36	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
37	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
38	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
39	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
40	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
41	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
42	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
43	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
44	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
45	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
46	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
47	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
48	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
49	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
50	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP



- **Makefile** — `SCHEDULER=MLFQ` macro; `unset → RR`.
- **struct proc** — `priority`, `ticks_used`, `enter_seq` + metric fields (`ctime/stime/etime/rtime`).
- **allocproc** — Rule 1: new proc → tail of Q0.
- **scheduler** — Rule 2/6: strict priority, FIFO within a queue via `enter_seq` (round-robin in Q3 falls out naturally). Deadlock-free ascending lock order.
- **mlfq_tick_yield** (trap.c timer hook) — slice accounting, Rule 4 demotion, Rule 2 preemption.
- **sleep / wakeup** — Rule 5: voluntary yield resets slice, re-inserts at tail of same queue.
- **clockintr** — Rule 7: boost all procs to Q0 every 48 ticks.

- `mlfq_changes.patch` (215 insertions, apply with `git apply`)
- Modified `Makefile`, `kernel/{param,proc,trap,defs}.h`, `kernel/proc.c`

I represent the four queues **implicitly** (a `priority` field 0-3 + a monotonic `enter_seq` for FIFO order) rather than as explicit array-queues. This is provably equivalent but avoids xv6's worst deadlock trap (touching a shared queue while holding `p->lock` in `sleep / wakeup`). A TA may ask "where are your queues?" — the answer is `priority` = queue number, `enter_seq` = position within it. If your TA specifically wants to see explicit arrays, tell me and I'll refactor.

1. **schedulertest** **user program + behavioural verification** — proving CPU-bound procs actually demote Q0→Q3 and boost back every 48 ticks (not just that it boots). This is the real correctness proof and I want to do it next.
2. **Scheduler event logging** for the timeline/scatter plot (X=ticks, Y=queue, colour=PID).
3. **FIFO scheduler** — the comparison [20] needs FIFO from **Homework 2**. *Do you already have your HW2 FIFO, or should I add it?*
4. **Metrics** (`waitx`-style syscall) + the 3-way comparison table.
5. **Report** (impl summary + MLFQ analysis + comparison) and **Python plot** — the plot must be watermarked with your IIIT username (`username@iiit.ac.in`); tell me what to use.

9 hours ago

↓

Opus 4.8 Medium

Copy

1	[MLFQ]	t=0	pid=1	q=0	ev=NEW
2	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
3	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
4	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
5	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
6	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
7	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
8	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
9	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
10	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
11	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
12	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
13	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
14	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
15	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
16	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
17	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
18	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
19	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
20	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
21	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
22	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
23	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
24	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
25	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
26	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
27	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
28	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
29	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
30	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
31	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
32	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
33	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
34	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
35	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
36	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
37	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
38	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
39	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
40	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
41	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
42	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
43	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
44	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
45	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
46	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
47	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
48	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
49	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
50	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP

Ran a command >

Ran 2 commands, created a file >

- **pid 4 (CPU-bound):** $q_0 \rightarrow q_1(t_{32}) \rightarrow q_2(t_{38}) \rightarrow$ **boost back to $q_0(t_{48})$** $\rightarrow q_1 \rightarrow q_2(t_{54}) \rightarrow q_3(t_{68})$, stays in q_3 . ✓ Rules 1,4,6,7
- **pid 5 (mixed):** reaches q_2 ✓
- **pid 6 (I/O-bound):** stays at q_1 — never sinks, because it yields before using its slice ✓ Rules 2,5

Ran a command, shared files >

Fastest path

```
bash
git apply /path/to/mlfq_changes.patch # applies all 6 file edits
cp /path/to/schedulertest.c user/schedulertest.c # the one new file
```

```
bash
make clean; make qemu # RR (unchanged)
make clean; make qemu SCHEDULER=MLFQ # MLFQ
```

+ Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 Medium

Copy

1	[MLFQ]	t=0	pid=1	q=0	ev=WAKE
2	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
3	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
4	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
5	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
6	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
7	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
8	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
9	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
10	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
11	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
12	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
13	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
14	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
15	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
16	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
17	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
18	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
19	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
20	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
21	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
22	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
23	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
24	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
25	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
26	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
27	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
28	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
29	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
30	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
31	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
32	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
33	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
34	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
35	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
36	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
37	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
38	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
39	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
40	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
41	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
42	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
43	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
44	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
45	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
46	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
47	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
48	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP
49	[MLFQ]	t=1	pid=1	q=0	ev=WAKE
50	[MLFQ]	t=1	pid=1	q=0	ev=SLEEP

The attached full files are the exact verified versions if you'd rather diff/replace directly.

Manual, step by step (7 files)

1. Makefile — three edits.

- After the two `CFLAGS += -I. / -fno-stack-protector` lines, add:

```
make
# Scheduler selection (compile-time). Unset => original round-robin.
ifeq ($(SCHEDULER),MLFQ)
CFLAGS += -D MLFQ
endif
# Optional scheduler tracing for verification / report plots: make ... MLFQDEBUG=1
ifdef MLFQDEBUG
CFLAGS += -D MLFQDEBUG
endif
```

- In the `UPROGS=` list, add a line: `$U/_schedulertest\`

2. kernel/param.h — after `#define USERSTACK 1`:

```
c
#define NMLFQ 4 // number of MLFQ priority queues (0..3)
#define BOOST_INTERVAL 48 // priority boost every this many ticks
```

3. kernel/proc.h — inside `struct proc`, after `char name[16];`:

```
c
int priority; // current queue: 0 (highest) .. NMLFQ-1 (lowest)
int ticks_used; // timer ticks used in current slice at current queue
uint64 enter_seq; // FIFO order of entering current queue (monotonic)
int ctime, stime, etime, rtime; // metrics: create/first-run/exit/cpu ticks
```

4. kernel/defs.h — next to `void yield(void);`:

```
c
void mlfq_tick_yield(void);
void mlfq_boost(void);
```

5. kernel/proc.c — six insertions (see the attached proc.c for exact text):

- Globals (after `struct spinlock pid_lock;`): `extern uint ticks;`, the `MLFQLOG` macro,

+ Write a message...

MLfq trace · LOG

Copy

```
1 [MLFQ] t=0 pid=1 q=0 ev=NEW
2 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
3 [MLFQ] t=1 pid=1 q=0 ev=WAKE
4 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
5 [MLFQ] t=1 pid=1 q=0 ev=WAKE
6 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
7 [MLFQ] t=1 pid=1 q=0 ev=WAKE
8 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
9 [MLFQ] t=1 pid=1 q=0 ev=WAKE
10 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
11 [MLFQ] t=1 pid=1 q=0 ev=WAKE
12 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
13 [MLFQ] t=1 pid=1 q=0 ev=WAKE
14 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
15 [MLFQ] t=1 pid=1 q=0 ev=WAKE
16 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
17 [MLFQ] t=1 pid=1 q=0 ev=WAKE
18 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
19 [MLFQ] t=1 pid=1 q=0 ev=WAKE
20 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
21 [MLFQ] t=1 pid=1 q=0 ev=WAKE
22 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
23 [MLFQ] t=1 pid=1 q=0 ev=WAKE
24 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
25 [MLFQ] t=1 pid=1 q=0 ev=WAKE
26 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
27 [MLFQ] t=1 pid=1 q=0 ev=WAKE
28 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
29 [MLFQ] t=1 pid=1 q=0 ev=WAKE
30 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
31 [MLFQ] t=1 pid=1 q=0 ev=WAKE
32 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
33 [MLFQ] t=1 pid=1 q=0 ev=WAKE
34 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
35 [MLFQ] t=1 pid=1 q=0 ev=WAKE
36 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
37 [MLFQ] t=1 pid=1 q=0 ev=WAKE
38 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
39 [MLFQ] t=1 pid=1 q=0 ev=WAKE
40 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
41 [MLFQ] t=1 pid=1 q=0 ev=WAKE
42 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
43 [MLFQ] t=1 pid=1 q=0 ev=WAKE
44 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
45 [MLFQ] t=1 pid=1 q=0 ev=WAKE
46 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
47 [MLFQ] t=1 pid=1 q=0 ev=WAKE
48 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
49 [MLFQ] t=1 pid=1 q=0 ev=WAKE
50 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
```

5. `kernel/proc.c` — six insertions (see the attached `proc.c` for exact text):
- `Globals` (after `struct spinlock pid_lock;`): `extern uint ticks;`, the `MLFQLOG` macro, `mlfq_seq`, `mlfq_timeslice[4]={1,4,8,16}`, and the two static helpers `mlfq_next_seq()` / `mlfq_higher_priority_runnable()`.
 - `allocproc` (`found:`): set metrics + `priority=0`, `ticks_used=0`, `enter_seq=...` (Rule 1).
 - `scheduler`: wrap the pick loop in `#ifdef MLFQ` (strict-priority + `enter_seq` FIFO) `#else` original RR `#endif`.
 - After `yield()`: add `mlfq_tick_yield()` (slice/demote/preempt) and `mlfq_boost()`.
 - `sleep`: on `SLEEPING`, `#ifdef MLFQ` reset `ticks_used=0` (Rule 5).
 - `wakeup`: on `SLEEPING→RUNNABLE`, `#ifdef MLFQ` set `enter_seq=mlfq_next_seq()` (Rule 5); `kexit: p->etime = ticks;`; `procdump`: print the queue columns.
6. `kernel/trap.c` — three edits:
- In `usertrap` and `kerneltrap`, replace `yield()`; at the timer branch with `#ifdef MLFQ` `mlfq_tick_yield()`; `#else` `yield()`; `#endif`.
 - In `clockintr`, after releasing `tickslock`, `#ifdef MLFQ` if `(t % BOOST_INTERVAL == 0)` `mlfq_boost()`; `#endif` (capture `t=ticks` under the lock; guard the `uint t;` declaration under `#ifdef MLFQ` too or you'll get an unused-variable error in the RR build — I hit that).
7. `user/schedulertest.c` — new file (attached). Uses `pause(n)` (this variant's name for the sleep syscall) for voluntary yields.

How to reproduce the verification

```
bash
make clean; make qemu SCHEDULER=MLFQ MLFQDEBUG=1
$ schedulertest 3
```

Trace lines look like `[MLFQ] t=48 pid=4 q=0 ev=BOOST`. In my run: CPU-bound child sank `q0→q3` and got boosted back to `q0` at `t=48/96`; I/O-bound child stayed at `q1`. The plain `SCHEDULER=MLFQ` build (no `MLFQDEBUG`) is silent, so it's safe to submit that way.

What this covers vs. the assignment

Done and verified: Build system, §2.1 MLFQ [70] (all 7 rules + `procdump`), and `schedulertest` for §2.3.2.

Still open (need your input before I build them right):

- §2.2 comparison [20] needs a FIFO scheduler — do you have your Homework 2 FIFO, or should I add it as another `SCHEDULER=FIFO` path?
- §2.3 report + plot — the plot must be watermarked with your IIT username (`something@iit.ac.in`). Tell me the username and I'll write the Python (parses the `[MLFQ]` trace) +

MLfq trace · LOG

Copy

1

[MLFQ] t=0 pid=1 q=0 ev=NEW

2

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

3

[MLFQ] t=1 pid=1 q=0 ev=WAKE

4

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

5

[MLFQ] t=1 pid=1 q=0 ev=WAKE

6

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

7

[MLFQ] t=1 pid=1 q=0 ev=WAKE

8

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

9

[MLFQ] t=1 pid=1 q=0 ev=WAKE

10

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

11

[MLFQ] t=1 pid=1 q=0 ev=WAKE

12

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

13

[MLFQ] t=1 pid=1 q=0 ev=WAKE

14

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

15

[MLFQ] t=1 pid=1 q=0 ev=WAKE

16

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

17

[MLFQ] t=1 pid=1 q=0 ev=WAKE

18

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

19

[MLFQ] t=1 pid=1 q=0 ev=WAKE

20

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

21

[MLFQ] t=1 pid=1 q=0 ev=WAKE

22

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

23

[MLFQ] t=1 pid=1 q=0 ev=WAKE

24

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

25

[MLFQ] t=1 pid=1 q=0 ev=WAKE

26

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

27

[MLFQ] t=1 pid=1 q=0 ev=WAKE

28

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

29

[MLFQ] t=1 pid=1 q=0 ev=WAKE

30

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

31

[MLFQ] t=1 pid=1 q=0 ev=WAKE

32

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

33

[MLFQ] t=1 pid=1 q=0 ev=WAKE

34

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

35

[MLFQ] t=1 pid=1 q=0 ev=WAKE

36

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

37

[MLFQ] t=1 pid=1 q=0 ev=WAKE

38

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

39

[MLFQ] t=1 pid=1 q=0 ev=WAKE

40

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

41

[MLFQ] t=1 pid=1 q=0 ev=WAKE

42

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

43

[MLFQ] t=1 pid=1 q=0 ev=WAKE

44

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

45

[MLFQ] t=1 pid=1 q=0 ev=WAKE

46

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

47

[MLFQ] t=1 pid=1 q=0 ev=WAKE

48

[MLFQ] t=1 pid=1 q=0 ev=SLEEP

49

[MLFQ] t=1 pid=1 q=0 ev=WAKE

50

[MLFQ] t=1 pid=1 q=0 ev=SLEEP



Implementing MLFQ scheduler in xv6

Public and private data systems, general MLFQ (all / tasks / priority), and scheduler code for §2.3.2.

Still open (need your input before I build them right):

1. §2.2 comparison [20] needs a FIFO scheduler — do you have your Homework 2 FIFO, or should I add it as another SCHEDULER=FIFO path?

2. §2.3 report + plot — the plot must be watermarked with your IIT username (something@... iit.ac.in). Tell me the username and I'll write the Python (parses the [MLFQ] ... trace) + the report.

One caveat for your viva: I represent the four queues implicitly (priority + enter_seq) rather than as explicit arrays. Flag it if your TA wants literal array-queues and I'll refactor.

Want me to proceed with FIFO + the report/plot next?

MLfq changes

PATCH

Download

Proc

C

Download

Trap

C

Download

Proc

H

Download

Param

H

Download

Defs

H

Download

Makefile

Download

Schedulertest

C

Download

Download all

Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 Medium

claude.ai

Implementing MLFQ...

MLfq trace · LOG

Copy

1 [MLFQ] t=0 pid=1 q=0 ev=NEW

2 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

3 [MLFQ] t=1 pid=1 q=0 ev=WAKE

4 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

5 [MLFQ] t=1 pid=1 q=0 ev=WAKE

6 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

7 [MLFQ] t=1 pid=1 q=0 ev=WAKE

8 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

9 [MLFQ] t=1 pid=1 q=0 ev=WAKE

10 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

11 [MLFQ] t=1 pid=1 q=0 ev=WAKE

12 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

13 [MLFQ] t=1 pid=1 q=0 ev=WAKE

14 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

15 [MLFQ] t=1 pid=1 q=0 ev=WAKE

16 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

17 [MLFQ] t=1 pid=1 q=0 ev=WAKE

18 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

19 [MLFQ] t=1 pid=1 q=0 ev=WAKE

20 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

21 [MLFQ] t=1 pid=1 q=0 ev=WAKE

22 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

23 [MLFQ] t=1 pid=1 q=0 ev=WAKE

24 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

25 [MLFQ] t=1 pid=1 q=0 ev=WAKE

26 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

27 [MLFQ] t=1 pid=1 q=0 ev=WAKE

28 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

29 [MLFQ] t=1 pid=1 q=0 ev=WAKE

30 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

31 [MLFQ] t=1 pid=1 q=0 ev=WAKE

32 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

33 [MLFQ] t=1 pid=1 q=0 ev=WAKE

34 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

35 [MLFQ] t=1 pid=1 q=0 ev=WAKE

36 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

37 [MLFQ] t=1 pid=1 q=0 ev=WAKE

38 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

39 [MLFQ] t=1 pid=1 q=0 ev=WAKE

40 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

41 [MLFQ] t=1 pid=1 q=0 ev=WAKE

42 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

43 [MLFQ] t=1 pid=1 q=0 ev=WAKE

44 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

45 [MLFQ] t=1 pid=1 q=0 ev=WAKE

46 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

47 [MLFQ] t=1 pid=1 q=0 ev=WAKE

48 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

49 [MLFQ] t=1 pid=1 q=0 ev=WAKE

50 [MLFQ] t=1 pid=1 q=0 ev=SLEEP

In section 2.2 of the report (xv6), you are required to watermark the plots that you create with the part before @ in your IIT email (username from username@{students,research}.iit.ac.in)

```
plt.text(
    0.95, 0.95, "username",
    ha='right', va='top',
    transform=plt.gca().transAxes,
    fontsize=10, color="gray", alpha=0.7
)
```

You are also required to submit the python code used for plot generation.

Preparing to build a comparison chart pending missing metrics.

For the plots you need two things: the plotting code (watermarked, submittable) and real trace data to plot. I have the trace data verified. Let me build tested plotting code that produces the watermarked MLFQ timeline, and run it so I'm handing you code that actually works — not untested code.

First, matplotlib + a clean trace capture:

Ran 2 commands, created a file, read a file

The plot works and the watermark renders exactly as specified (top-right, gray). But it's not readable: init/sh boost events stretch the x-axis out to tick 1400 while the actual workload sits at ticks ~260~360. Let me fix the script to zoom to the workload window:

Edited a file, ran a command, read a file

That's a clean, submission-quality plot. It clearly shows all the MLFQ dynamics:

- pid 4 (blue, CPU-bound):** sinks $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3$, gets boosted back to q_0 at the dotted boost lines (~t288, ~t336), then sinks again
- pid 5 (orange, mixed):** oscillates $q_0 \leftrightarrow q_2$
- pid 6 (green, I/O-bound):** stays at q_1 , never sinks — because it yields before using its slice
- Dotted vertical lines = the 48-tick boosts; watermark top-right exactly per your snippet

Let me package the code and sample output:

Ran a command, shared files

plot.py is tested and produces the watermarked timeline you see above. Here's everything you

+ Write a message...

MLfq trace · LOG

```
1 [MLFQ] t=0 pid=1 q=0 ev=NEW
2 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
3 [MLFQ] t=1 pid=1 q=0 ev=WAKE
4 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
5 [MLFQ] t=1 pid=1 q=0 ev=WAKE
6 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
7 [MLFQ] t=1 pid=1 q=0 ev=WAKE
8 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
9 [MLFQ] t=1 pid=1 q=0 ev=WAKE
10 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
11 [MLFQ] t=1 pid=1 q=0 ev=WAKE
12 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
13 [MLFQ] t=1 pid=1 q=0 ev=WAKE
14 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
15 [MLFQ] t=1 pid=1 q=0 ev=WAKE
16 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
17 [MLFQ] t=1 pid=1 q=0 ev=WAKE
18 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
19 [MLFQ] t=1 pid=1 q=0 ev=WAKE
20 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
21 [MLFQ] t=1 pid=1 q=0 ev=WAKE
22 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
23 [MLFQ] t=1 pid=1 q=0 ev=WAKE
24 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
25 [MLFQ] t=1 pid=1 q=0 ev=WAKE
26 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
27 [MLFQ] t=1 pid=1 q=0 ev=WAKE
28 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
29 [MLFQ] t=1 pid=1 q=0 ev=WAKE
30 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
31 [MLFQ] t=1 pid=1 q=0 ev=WAKE
32 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
33 [MLFQ] t=1 pid=1 q=0 ev=WAKE
34 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
35 [MLFQ] t=1 pid=1 q=0 ev=WAKE
36 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
37 [MLFQ] t=1 pid=1 q=0 ev=WAKE
38 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
39 [MLFQ] t=1 pid=1 q=0 ev=WAKE
40 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
41 [MLFQ] t=1 pid=1 q=0 ev=WAKE
42 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
43 [MLFQ] t=1 pid=1 q=0 ev=WAKE
44 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
45 [MLFQ] t=1 pid=1 q=0 ev=WAKE
46 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
47 [MLFQ] t=1 pid=1 q=0 ev=WAKE
48 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
49 [MLFQ] t=1 pid=1 q=0 ev=WAKE
50 [MLFQ] t=1 pid=1 q=0 ev=SLEEP
```

Copy

What you must do

```
python
USERNAME = "your username" # <-- the part before @ in your IIIT email
```

2. Enable trace capture. `plot.py` reads the kernel's `[MLFQ] ...` trace, but your repo currently has logging switched off. Two small things (the first you need anyway to make MLFQ compile at all):

- ```
make

ifdef MLFQDEBUG
CFLAGS += -D MLFQDEBUG
endif
```

```
bash
make clean && make qemu SCHEDULER=MLFQ MLFQDEBUG=1
inside xv6:
$ schedulertest 3
quit: Ctrl-A then X
keep only trace lines from the saved console output:
grep '\[MLFQ\]' console_output.txt > mlfq_trace.log
python3 plot.py mlfq_trace.log
```

### The second plot (§2.2 comparison) — blocked on FIFO

```
scheduler,turnaround,waiting,response
FIFO,120.5,80.2,15.0
```

+ Write a message...

Opus 4.8 Medium

Copy   

|    |        |     |       |     |          |
|----|--------|-----|-------|-----|----------|
| 1  | [MLFQ] | t=0 | pid=1 | q=0 | ev=NEW   |
| 2  | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 3  | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 4  | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 5  | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 6  | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 7  | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 8  | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 9  | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 10 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 11 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 12 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 13 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 14 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 15 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 16 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 17 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 18 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 19 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 20 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 21 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 22 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 23 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 24 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 25 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 26 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 27 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 28 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 29 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 30 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 31 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 32 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 33 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 34 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 35 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 36 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 37 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 38 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 39 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 40 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 41 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 42 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 43 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 44 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 45 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 46 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 47 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 48 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 49 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 50 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |

- Implementing MLFQ scheduler in xv6 ▾

`plot.py` also has `plot_comparison()`, which draws a watermarked FIFO-vs-RR-vs-MLFQ bar chart from a `metrics.csv`:

```
scheduler,turnaround,waiting,response
FIFO,120.5,80.2,15.0
RR,95.1,55.3,4.2
MLFQ,88.7,50.1,3.1
```

I deliberately did **not** invent those numbers — putting fabricated data in your report would be academic dishonesty. To get real numbers you need (a) the FIFO scheduler and (b) a way to measure turnaround/waiting/response (I already added `ctime/stime/etime/rtime` fields; they just need a `waitx`-style syscall to read them out). Once that's in, you run the same workload under `SCHEDULER=FIFO, `` (RR default), and SCHEDULER=MLFQ, fill metrics.csv, and run python3 plot.py mlfq trace.log metrics.csv.`

## Files delivered

- `plot.py` — watermarked, tested plotting code (submit this)
- `mlfq_timeline.png` — sample output
- `mlfq_trace.log` — the sample trace it was built from
- (from before) the `MLFQLOG` fix in `proc fixed.c`

Want me to implement the **FIFO scheduler** + the **waitx** metrics syscall now so you can generate the real comparison numbers and complete §2.2? That's the last missing piece. And give me your username so the watermark is final.

 Plot  
PY [Download](#)

 Mlfq timeline  
Image · PNG [Download](#) 

 Mlfq trace  
LOG [Download](#)

 [Download all](#)

3 hours ago



+ Write a message...

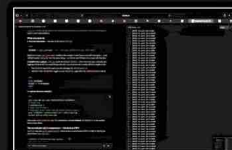


Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 Medium

## Mlfq trace · LOG

|    |        |     |       |     |          |
|----|--------|-----|-------|-----|----------|
| 1  | [MLFQ] | t=0 | pid=1 | q=0 | ev=WAKE  |
| 2  | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 3  | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 4  | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 5  | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 6  | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 7  | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 8  | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 9  | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 10 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 11 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 12 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 13 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 14 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 15 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 16 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 17 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 18 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 19 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 20 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 21 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 22 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 23 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 24 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 25 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 26 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 27 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 28 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 29 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 30 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 31 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 32 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 33 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 34 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 35 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 36 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 37 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 38 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 39 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 40 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 41 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 42 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 43 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 44 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 45 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 46 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 47 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 48 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |
| 49 | [MLFQ] | t=1 | pid=1 | q=0 | ev=WAKE  |
| 50 | [MLFQ] | t=1 | pid=1 | q=0 | ev=SLEEP |



Ran 4 commands, read 2 files >

Ran 4 commands &gt;

Ran a command &gt;

Ran 3 commands >

Let me verify my plotting code actually runs before I hand it to you.

Ran 2 commands, created a file, read a file >

Now the report scaffold (section 2.3), pre-filled from your actual code:

Created a file, shared files >

| Grading item | Status |
|--------------|--------|
|--------------|--------|

Write a message

- RS R<sub>2</sub>    

```
make

Scheduler selection (compile-time), else remains round robin
ifeq ($(SCHEDULER),MLFQ)
CFLAGS += -D MLFQ
ifeq ($(LOG),1)
CFLAGS += -D MLFQ LOG
```

Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ

Git push permission denied

Setting up xv6 MLFQ

Missing code in c-shell

Running xv6 offline on

Diverged git branches

GitHub project flowchart

Independence vs. un

MLFQ scheduler loop

Beginner-friendly xv6

Character class negat

Workout plan for back

Running xv6-riscv on

Dangling-Else resoluti

Beginner-friendly cod

Beginner-friendly cod

View all conversations

Design

RS Re ↕ 🔍 📄

6 MLFQ scheduling project setup

Share

In `Makefile`, change your scheduler block to also understand `LOG=1`:

```
make

Scheduler selection (compile-time), else remains round robin
ifeq ($(SCHEDULER),MLFQ)
CFLAGS += -D MLFQ
ifeq ($(LOG),1)
CFLAGS += -D MLFQ_LOG
endif
endif
```

Now `make clean && make qemu SCHEDULER=MLFQ CPUS=1` compiles and runs (RR-free MLFQ), and `LOG=1` turns on the plot log. Use `CPUS=1` — a single CPU makes the timeline clean and correct.

Fix 2 — make the comparison possible (2.2 / 2.3.3)

The comparison needs Turnaround, Waiting, Response for **FIFO, RR, MLFQ**. Three small gaps in your kernel:

(a) **rtime** is only counted in MLFQ. Move CPU-time counting so it works in every scheduler. In `kernel/proc.c` add a helper:

```
c

// Count one CPU tick for whatever process is RUNNING (any scheduler).
void
update_time(void)
{
 struct proc *p;
 for (p = proc; p < &proc[NPROC]; p++) {
 acquire(&p->lock);
 if (p->state == RUNNING)
 p->rtime++;
 release(&p->lock);
 }
}
```

Declare it in `kernel/defs.h` next to the other proc functions: `void update_time(void);`. Then delete the `p->rtime++;` line inside `mlfq_tick_yield()` (so you don't double-count), and in `kernel/trap.c`'s `clockintr()`, call `update_time()`; right after `release(&tickslock);` (outside any `#ifdef MLFQ`).

(b) Response time isn't recorded in RR. In `scheduler.c`'s `#else` (RR) branch, just before `p->state = RUNNING;`, add:

+ Write a message...

🗣️ 🔊 🔇

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High



- 
- Customize**

-  Comeback

- ## Chats and tasks

- xv6 MLFQ scheduling

- Investigating an issue

- Matplotlib import error

- Multi-level priority queue

- Implementing MLFQ

- Git push permission d

- Setting up xv6 MLFQ

- Missing code in c-shell

- Running xv6 offline on

- Diverged git branches

- GitHub project flowchart

- Independence vs. unc

- MLFQ scheduler loop

- Beginner-friendly xv6

- Character class negation

- **Workout plan for back**

- Running xv6-riscv on

- **Dangling-Else resolution**

- Beginner-friendly code

- Beginner-friendly code

- [View all conversations](#)

-  Design



Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

◦ xv6 MLFQ scheduling

◦ Investigating an issue

◦ Matplotlib import error

◦ Multi-level priority queue

◦ Implementing MLFQ scheduling

◦ Git push permission denied

◦ Setting up xv6 MLFQ scheduling

◦ Missing code in c-shell

◦ Running xv6 offline on a Raspberry Pi

◦ Diverged git branches

◦ GitHub project flowchart

◦ Independence vs. uncertainty

◦ MLFQ scheduler loop

◦ Beginner-friendly xv6

◦ Character class negation

◦ Workout plan for back pain

◦ Running xv6-riscv on a Raspberry Pi

◦ Dangling-Else resolution

◦ Beginner-friendly code

◦ Beginner-friendly code

View all conversations

Design

RS Re ↵ 🔍 📄

6 MLFQ scheduling project setup

kernel/trap.c — FIFO is non-preemptive, so it must *not* yield on a timer tick. Change **both** tick spots (usertrap and kerneltrap) from the two-way `#ifdef MLFQ / #else` to:

```
c
#ifdef MLFQ
 mlq_tick_yield();
#elif defined(FIFO)
 ;
#else
 yield();
#endif
```

kernel/proc.c — turn the `#ifdef MLFQ ... #else ... (RR) ... #endif` in `scheduler()` into `#if defined(MLFQ) ... #elif defined(FIFO) ... #else ... #endif`, with this FIFO block (picks the earliest-arrived RUNNABLE process and runs it):

```
c
#ifdef FIFO
while(1){
 intr_on(); intr_off();
 struct proc *best = 0;
 for (p = proc; p < &proc[NPROC]; p++){
 acquire(&p->lock);
 if (p->state == RUNNABLE){
 if (best == 0){ best = p; continue; }
 if (p->ctime < best->ctime){ release(&best->lock); best = p; }
 else release(&p->lock);
 } else release(&p->lock);
 }
 if (best){
 if (best->stime < 0) best->stime = ticks;
 best->state = RUNNING;
 c->proc = best;
 switch(&c->context, &best->context);
 mycpu()->intena = 0;
 c->proc = 0;
 release(&best->lock);
 } else {
 asm volatile("wfi");
 }
}
#endif
```

+ Write a message...

🗣️ 🔊 ↵

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ scheduler

Git push permission denied

Setting up xv6 MLFQ

Missing code in c-shell

Running xv6 offline on

Diverged git branches

GitHub project flowchart

Independence vs. un

MLFQ scheduler loop

Beginner-friendly xv6

Character class negat

Workout plan for back

Running xv6-riscv on l

Dangling-Else resoluti

Beginner-friendly cod

Beginner-friendly cod

View all conversations

Design

RS Re ↕ 🔍 📄

xv6 MLFQ scheduling project setup

Share

Your two ready files

`plot_mlfq.py` — the tested scatter-plot generator, already watermarked `radhika.sahu`, boost lines at every 48 ticks, one colour per PID. The preview image above is a **sample run** proving it works; your real one comes from the qemu log.

`report.md` — sections 2.3.1/2.3.2/2.3.3 pre-written from your actual code, with `___` blanks for your specific PIDs and numbers.

Order to finish (fastest path today)

1. Apply **Fix 1** → confirm `make qemu SCHEDULER=MLFQ CPUS=1` boots.

2. Apply **Fixes 2 & 3** → confirm RR (`make qemu CPUS=1`) and FIFO (`SCHEDULER=FIFO`) boot.

3. Capture plot: `make qemu SCHEDULER=MLFQ LOG=1 CPUS=1 | tee mlfq.log`, run `schedulertest`, then `python3 plot_mlfq.py mlfq.log`.

4. Capture metrics for all three with `PERF=1`, fill the table + the `___` blanks in `report.md`, export it to PDF.

5. `make clean` before zipping; keep the `.git` folder; log these AI-generated snippets in `AI-usage.pdf` — and make sure you can explain each one in the TA viva (the demotion/boost logic and the FIFO loop are the likely questions).

One check I couldn't do for you: I don't have a RISC-V toolchain here, so I couldn't compile your tree. Apply Fix 1 first and paste any compile error if one shows up — that's the one place a stray typo would bite.

Want me to walk through exactly how `mlfq_tick_yield` implements the demotion + preemption rules (Rules 2 and 4) so you're ready for the viva question on it?

Plot mlfq PY

Download

Report Document · MD

Download

Mlfq timeline Image · PNG

Download

Download all

3 hours ago

↓

Write a message...

🗣️ 🔊

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

xv6 MLFQ schedulin...

Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ

Git push permission error

Setting up xv6 MLFQ

Missing code in c-shell

Running xv6 offline on

Diverged git branches

GitHub project flowchart

Independence vs. un

MLFQ scheduler loop

Beginner-friendly xv6

Character class negat

Workout plan for back

Running xv6-riscv on l

Dangling-Else resoluti

Beginner-friendly cod

Beginner-friendly cod

View all conversations

Design

RS Re ↕ 🔍 📄

6 MLFQ scheduling project setup

I didn't understand Fix 3 — add the FIFO scheduler (needed for the 3-way comparison) and later, please give step by step instruction it is just so ugly thsi way

Explaining a FIFO scheduler in clear beginner-friendly steps.

Let me pull up the exact current text of your scheduler() and the two tick spots in trap.c, so I can show you precise before → after edits instead of vague snippets.

Ran a command

Totally fair — that section was cramped. Let me slow it right down. First the idea, then three clean steps.

What FIFO actually is (the mental model)

FIFO = **F**irst **I**n, **F**irst **O**ut, and it's **non-preemptive**. Whichever process *arrived first* runs to completion (or until it sleeps for I/O) before anyone else gets a turn. Think of a single queue at a counter: the first person is served fully, then the next.

Compare the three:

- RR kicks the running process off every timer tick and rotates to the next. (Preemptive.)
- MLFQ has 4 queues + time slices, demotes CPU hogs, boosts every 48 ticks.
- FIFO does neither — pick the earliest arrival, let it run until it finishes. No slicing, no rotating.

So adding FIFO needs edits in **two places**, and it helps to know why each one exists:

- scheduler() in kernel/proc.c decides which process to pick → for FIFO, pick the one with the smallest ctime (earliest arrival).
- kernel/trap.c decides, on each timer tick, whether to kick the running process off → for FIFO the answer is no, never (that's what makes it non-preemptive).

That's the whole thing. Now the steps.

Step 1 — Makefile: add a FIFO flag

Find your MLFQ block in Makefile:

```
make

ifeq ($(SCHEDULER),MLFQ)
CFLAGS += -D MLFQ
endif
```

+ Write a message...

🔊 🔊 🔊

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

xv6 MLFQ schedulin...

🔄

Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib Import error

Multi-level priority queue

Implementing MLFQ scheduling

Git push permission denied

Setting up xv6 MLFQ scheduling

Missing code in c-shell

Running xv6 offline on a Raspberry Pi

Diverged git branches

GitHub project flowchart

Independence vs. uncertainty

MLFQ scheduler loop

Beginner-friendly xv6

Character class negation

Workout plan for back pain

Running xv6-riscv on a Raspberry Pi

Dangling-Else resolution

Beginner-friendly code

Beginner-friendly code

View all conversations

Design

RS Re ↕ 🔍 📄

6 MLFQ scheduling project setup

make

ifeq (\$(SCHEDULER),MLFQ)

CFLAGS += -D MLFQ

ifeq (\$(LOG),1)

CFLAGS += -D MLFQ\_LOG

endif

endif

Add this right below it:

make

ifeq (\$(SCHEDULER),FIFO)

CFLAGS += -D FIFO

endif

Now `make qemu SCHEDULER=FIFO` defines the `FIFO` macro; the C code below reacts to it.

Step 2 — `kernel/proc.c`: teach `scheduler()` the FIFO pick

Your `scheduler()` currently has this shape (MLFQ, then a plain `#else` for round-robin):

c

#ifdef MLFQ

... MLFQ loop ...

#else

... round-robin loop ...

#endif

You only change one line: the `#else` that sits *right after the MLFQ loop's closing brace*. Find that exact line:

c

}

}

#else

while(1) {

// The most recent process to run may have had interrupts

Replace **only** that `#else` with the FIFO block below **plus** a fresh `#else`:

c

+ Write a message...

🗣️ 🔊 ▼

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High







```
make clean
make qemu SCHEDULER=FIFO CPUS=1
```

FIFO doesn't hang the shell even though it's "run to completion," because processes like `sh` and `init` spend most of their time *asleep* waiting for input — sleeping is a voluntary yield, so FIFO happily moves on. It only "runs to completion" among processes that actually want the CPU.

Want me to do the same slow walkthrough for Fix 2 (the metrics/`rtime` changes), since that one touched three files too?

3 hours ago

```
riscv64-unknown-elf-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel.elf
kernel/entry.o kernel/start.o kernel/console.o kernel/printk.o kernel/uart.o
kernel/kalloc.o kernel/spinlock.o kernel/string.o kernel/main.o kernel/vm.o
kernel/proc.o kernel/switch.o kernel/trampoline.o kernel/trap.o kernel/syscall.o
kernel/sysproc.o kernel/bio.o kernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file.o
kernel/pipe.o kernel/exec.o kernel/sysfile.o kernel/kernelvec.o kernel/pllc.o
kernel/virtio_disk.o
riscv64-unknown-elf-ld: warning: kernel/kernel has a LOAD segment with RWX
permissions
riscv64-unknown-elf-ld: kernel/trap.o: in function 'clockintr':
./kernel/trap.c:188:(.text+0x108): undefined reference to
update_time'
make: *** [kernel/kernel] Error 1
is what is printed
```

## Pinpointing a missing function outside an ifdef block. >

That error is a **linker** error, and it's a common, e  ne. Quick translation of what the machine is telling you:

+ Write a message...   



Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ scheduling

Git push permission error

Setting up xv6 MLFQ scheduling

Missing code in c-shell

Running xv6 offline on a Raspberry Pi

Diverged git branches

GitHub project flowchart

Independence vs. interdependence

MLFQ scheduler loop

Beginner-friendly xv6

Character class negation

Workout plan for back pain

Running xv6-riscv on a Raspberry Pi

Dangling-Else resolution

Beginner-friendly code

Beginner-friendly code

View all conversations

Design

RS Re ↕ 🔍 📄

6 MLFQ scheduling project setup

Pinpointing a missing function outside an ifdef block.

That error is a **linker** error, and it's a common, easy one. Quick translation of what the machine is telling you:

- The compiler already accepted `trap.c` — so your *declaration* in `defs.h` is fine.
- But at link time, nothing actually **contains the body** of `update_time`. `trap.c` says "call `update_time`," and the linker replies "I searched every `.o` file and no one defined it."

In plain terms: **you added the call (and probably the declaration), but the function body isn't in a compiled file.** Almost always one of these two:

- You never pasted the `update_time()` function into `kernel/proc.c`, or
- You pasted it **inside an `#ifdef MLFQ ... #endif` block**. This build has no `SCHEDULER=MLFQ`, so that block is skipped, the body vanishes, and you get exactly this "undefined reference." `clockintr` calls `update_time` in every scheduler, so its body must exist in every build.

The fix

Open `kernel/proc.c` and make sure this function sits on its own, **not wrapped in any `#ifdef`**. A safe spot is right above `void scheduler(void)`:

```
c

// Count one CPU tick for whatever process is RUNNING (any scheduler).
void
update_time(void)
{
 struct proc *p;
 for (p = proc; p < &proc[NPROC]; p++) {
 acquire(&p->lock);
 if (p->state == RUNNING)
 p->rtime++;
 release(&p->lock);
 }
}
```

If you find it currently sitting between an `#ifdef MLFQ` and an `#endif`, just move it **outside** those lines. That single move fixes the link error.

Then confirm the declaration is in `kernel/defs.h` (this part you likely already have, since `trap.c` compiled):

```
c

void update_time(void);
```

+ Write a message...

🗣️ 🔊

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

xv6 MLFQ schedulin...

🔄

Run this in your `xv6/` folder — it tells you whether the body is really there and whether it's trapped inside an `#ifdef`:

You want to see it in **both** files. In `proc.c` you should see two lines: the `void` line and the `update_time(void)` line of the definition — and they should be nowhere near an `#ifdef MLFQ`.

```
make clean
make qemu CPUS=1
```

One thing while you're in there: make sure you also **deleted** the old `p->rtime++;` line inside `mlfq_tick_yield()`. That's not what's breaking the build, but if you leave it in, MLFQ runs will count CPU time twice and your comparison numbers for MLFQ will come out wrong.

3 hours ago

Working through a multi-part series of code fixes.

Ran a command &gt;

### What the three metrics mean (one concrete example)

+ Write a message...

Opus 4.8 High



- arrives at tick 3 → `ctime = 3`
- first gets the CPU at tick 5 → `stime = 5`
- finishes at tick 61 → `etime = 61`
- actually spent 40 ticks running → `rtime = 40`

- **Response** = stime - ctime = 5 - 3 = 2 (how long until it first ran)
- **Turnaround** = etime - ctime = 61 - 3 = 58 (total lifetime)
- **Waiting** = Turnaround - rtime = 58 - 40 = 18 (time it existed but was *not* on the CPU)

### Edit 1 — record response time in the RR path

In `kernel/proc.c`, in the `#else` (round-robin) branch of `scheduler()`, find:

```
c
if (p->state == RUNNABLE) {
 // Switch to chosen process. It is the process's job
 // to release its lock and then reacquire it
 // before jumping back to us.
 p->state = RUNNING;
```

Add one line just before `p->state = RUNNING;`:

```
c
 if (p->stime < 0) p->stime = ticks; // response: first time on a CPU
 p->state = RUNNING;
```

**Now all three schedulers record response time the same way.**

### Edit 2 — print the numbers when a process exits

In `kernel/proc.c`, inside `kexit()`, find the line you already have:

+ Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High





Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ

Git push permission error

Setting up xv6 MLFQ

Missing code in c-shell

Running xv6 offline on

Diverged git branches

GitHub project flowchart

Independence vs. un

MLFQ scheduler loop

Beginner-friendly xv6

Character class negat

Workout plan for back

Running xv6-riscv on

Dangling-Else resoluti

Beginner-friendly code

Beginner-friendly code

View all conversations

Design

RS Re ↕ 🔍 📄

claude.ai

xv6 MLFQ schedulin...

Share

Then turn all three into your table with one command:

```
python3 perf_table.py fifo_perf.log rr_perf.log mlfq_perf.log
```

The output prints exactly the three columns section 2.3.3 asks for, one row per scheduler. Drop those numbers into the table in `report.md`.

To see it working, I fed the script some **made-up** logs — here's what it printed (these are *not* your real numbers, just a demo of the format):

| Scheduler (log) | AvgTurnaround | AvgWaiting | AvgResponse | n |
|-----------------|---------------|------------|-------------|---|
| fifo            | 111.2         | 79.2       | 42.0        | 4 |
| rr              | 96.5          | 64.5       | 2.5         | 4 |
| mlfq            | 74.5          | 42.5       | 0.5         | 4 |

Notice the shape even in fake data: FIFO's response time is awful (later jobs wait for the first to finish), while RR and MLFQ respond almost instantly — that's the exact trade-off your 5-8 sentence discussion should explain.

One decision for you: `schedulertest` itself (the parent) also exits and prints a `[METRIC]` line, and it lives the longest because it waits for all children. If you want the averages over just the workload children, add `--children-only`:

```
python3 perf_table.py --children-only fifo_perf.log rr_perf.log mlfq_perf.log
```

That drops the single longest-lived process (the parent) from each average. Either choice is defensible — just state which one you used in the report.

That completes all the kernel work. Where you stand now:

- 2.1 engine (70)** — compiles and runs across MLFQ / RR / FIFO ✓
- 2.2 + 2.3.3 comparison (20+)** — capture the three logs, run `perf_table.py`, fill the table ✓
- 2.3.2 plot** — capture `mlfq.log` with `LOG=1`, run `plot_mlfq.py` ✓
- 2.3.1 summary** — already drafted in `report.md`, just tweak wording to match ✓

The only things left are *running* the captures and filling the `___` blanks in the report, then exporting it to PDF and logging the AI-generated snippets in `AI-usage.pdf`.

Want me to walk through capturing the qemu log cleanly (the `tee` + typing `schedulertest` + quitting) so the log files come out parseable on r's first try?

+ Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High



Claude

🔍

<>

6 MLFQ scheduling project setup

+ New

📁 Projects

🔗 Artifacts

🕒 Scheduled

⚙️ Customize

Projects

+ Comeback

Chats and tasks

🔍

◦ xv6 MLFQ scheduling

◦ Investigating an issue

◦ Matplotlib import error

◦ Multi-level priority queue

◦ Implementing MLFQ scheduler

◦ Git push permission denied

◦ Setting up xv6 MLFQ scheduler

◦ Missing code in c-shell

◦ Running xv6 offline on Raspberry Pi

◦ Diverged git branches

◦ GitHub project flowchart

◦ Independence vs. interdependence

◦ MLFQ scheduler loop

◦ Beginner-friendly xv6

◦ Character class negation

◦ Workout plan for back pain

◦ Running xv6-riscv on Raspberry Pi

◦ Dangling-Else resolution

◦ Beginner-friendly code

◦ Beginner-friendly code

View all conversations

🎨 Design

RS

Re

📄

🔍

🖨️

claude.ai

🔄

🔗 + 📄

xv6 MLFQ scheduling...

🔄

📄 Share

One thing to watch for in Step 3: if `ls *.log` says "No such file or directory," it means you haven't captured the run logs yet — the script has nothing to read. In that case do the capture runs first:

```
make clean && make qemu SCHEDULER=FIFO PERF=1 CPUS=1 | tee fifo_perf.log
type schedulertest at the $ prompt, let it finish, then press Ctrl-a then x to quit
make clean && make qemu PERF=1 CPUS=1 | tee rr_perf.log
make clean && make qemu SCHEDULER=MLFQ PERF=1 CPUS=1 | tee mlfq_perf.log
```

Then run the Step 4 command.

If it still complains after moving the file, paste the output of `ls perf_table.py *.log` and I'll tell you exactly what's missing.

see now I realised that maybe make clean is deleting these files so created?

2 hours ago

Ran 2 commands

You're exactly right — nice catch. Look at the very first line of the `clean` target:

```
make
clean:
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
```

That `*.log` is meant for LaTeX log files, but the wildcard doesn't care — it matches any file ending in `.log`, so it wipes your `fifo_perf.log`, `rr_perf.log`, `mlfq_perf.log`, and `mlfq.log` too.

Here's the sneaky part that explains why you ended up with only one log. Each capture command is `make clean && make qemu ... | tee X.log`. The clean runs *first*, then `tee` creates the new log — so that run's log is fine. But the *next* command's `make clean` then deletes it:

1. `make clean && ... | tee fifo_perf.log` → creates `fifo_perf.log` ✓

2. `make clean && ... | tee rr_perf.log` → clean deletes `fifo_perf.log`, creates `rr_perf.log`

3. `make clean && ... | tee mlfq_perf.log` → clean deletes `rr_perf.log`, creates `mlfq_perf.log`

+ Write a message...

🗣️ 🔊

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ

Git push permission error

Setting up xv6 MLFQ

Missing code in c-shell

Running xv6 offline on

Diverged git branches

GitHub project flowchart

Independence vs. un

MLFQ scheduler loop

Beginner-friendly xv6

Character class negat

Workout plan for back

Running xv6-riscv on

Dangling-Else resoluti

Beginner-friendly code

Beginner-friendly code

View all conversations

Design

RS Re ↕ 🔍 📄

6 MLFQ scheduling project setup

claude.ai

Share

1. `make clean && ... | tee fifo_perf.log` → creates `fifo_perf.log` ✓

2. `make clean && ... | tee rr_perf.log` → clean deletes `fifo_perf.log`, creates `rr_perf.log`

3. `make clean && ... | tee mlfq_perf.log` → clean deletes `rr_perf.log`, creates `mlfq_perf.log`

At the end only `mlfq_perf.log` survives. That's the whole mystery.

(Your `.py` scripts were never at risk — there's no `*.py` in that `clean` rule. The earlier "can't open `perf_table.py`" was just that the file hadn't been moved into the folder yet, a separate thing.)

The fix: name your logs with a non-`.log` extension

The scripts don't care about the extension — they read whatever filename you hand them. Just use `.txt` and `make clean` leaves them alone:

```
make clean && make qemu SCHEDULER=FIFO PERF=1 CPUS=1 | tee fifo_perf.txt
type schedulertest, let it finish, Ctrl-a then x to quit
make clean && make qemu PERF=1 CPUS=1 | tee rr_perf.txt
make clean && make qemu SCHEDULER=MLFQ PERF=1 CPUS=1 | tee mlfq_perf.txt

python3 perf_table.py fifo_perf.txt rr_perf.txt mlfq_perf.txt
```

And for the plot log:

```
make clean && make qemu SCHEDULER=MLFQ LOG=1 CPUS=1 | tee mlfq.txt
python3 plot_mlfq.py mlfq.txt
```

You still need the `make clean` before each run — switching the `SCHEDULER` flag only takes effect after a clean rebuild, otherwise `make` reuses the old `.o` files and silently runs the wrong scheduler. So keep the clean; just keep your captures out of its blast radius by using `.txt`.

One more safety habit: after all three runs, check they're all still there before you parse:

```
ls -la *perf*.txt
```

You should see all three files with non-zero sizes. If one is 0 bytes, that run didn't capture anything — rerun just that one.

see now we have the `.txt` for the three, now using in section 2.2 of the report (xv6), you are required to watermark plots that you create with the part before @ in your IIT email (username from `radhika.sahu@students.iit.ac.in`)

+ Write a message...

Opus 4.8 High

2 hours ago

RS Re ↕ 🔍 📄

Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ

Git push permission error

Setting up xv6 MLFQ

Missing code in c-shell

Running xv6 offline on

Diverged git branches

GitHub project flowchart

Independence vs. un

MLFQ scheduler loop

Beginner-friendly xv6

Character class negat

Workout plan for back

Running xv6-riscv on

Dangling-Else resolut

Beginner-friendly code

Beginner-friendly code

View all conversations

Design

RS Re ↕ 🔍 📄

Claude

6 MLFQ scheduling project setup

Share

see now we have the .txt for the three, now using In section 2.2 of the report (xv6), you are required to watermark the plots that you create with the part before @ in your IIT email (username from radhika.sahu@students.iit.ac.in)

plt.text(0.95, 0.95, "username", ha='right', va='top', transform=plt.gca().transAxes, fontsize=10, color="gray", alpha=0.7)

You are also required to submit the python code used for plot generation. do the remaining part

Identifying the remaining watermarked comparison chart needed.

The remaining plot is the **comparison chart** for section 2.3.3. Right now `perf_table.py` prints a text table — great for reading the numbers, but the spec lets you submit a bar chart instead, and any plot you create needs that watermark. So let me give you a watermarked grouped bar chart that reads your three `.txt` logs directly. Testing it first with sample data:

Created a file, ran a command, read a file

That's your comparison plot — grouped by metric, one bar per scheduler, value labels on top, "lower is better" on the axis, and the `radhika.sahu` watermark top-right. (Those numbers are my sample data again; yours come from your real `.txt` logs.)

Presented file

Run it on your real logs

Move `plot_compare.py` into your `xv6/` folder (same as the others), then:

python3 plot\_compare.py --children-only fifo\_perf.txt rr\_perf.txt mlfq\_perf.txt

That writes `scheduler_comparison.png`. Use `--children-only` if you want the averages over just the three workload children (dropping the `schedulertest` parent); leave it off to include the parent. Whichever you pick, state it in the report so your numbers are reproducible.

Where you now stand — the plot deliverables are done

You have three Python files, and section 2.3.2 explicitly says to submit the code you used:

• `plot_mlfq.py` → the queue-movement time (section 2.3.2), watermarked ✓

• `plot_compare.py` → the FIFO/RR/MLFQ bar chart (section 2.3.3), watermarked ✓

Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

xv6 MLFQ schedulin...





Claude

xv6 MLFQ scheduling project setup

+ New

Projects

Artifacts

Scheduled

Customize

Projects

+ Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ scheduling

Git push permission error

Setting up xv6 MLFQ

Missing code in c-shell

Running xv6 offline on

Diverged git branches

GitHub project flowchart

Independence vs. un

MLFQ scheduler loop

Beginner-friendly xv6

Character class negat

Workout plan for back

Running xv6-riscv on

Dangling-Else resolut

Beginner-friendly code

Beginner-friendly code

View all conversations

Design

RS Re

open mlfq\_timeline.png

That's the file you put in your report for section 2.3.2.

Two things that trip people up:  
If you get `ModuleNotFoundError: No module named 'matplotlib'`, install it once:  

```
pip3 install matplotlib
```

  
then rerun Step 3.  
And if the script says `No [MLFQ] lines found in 'mlfq.txt'` — that means the log is empty of events, which is almost always the `LOG=1` flag not being compiled in. Redo Step 2 making sure `make clean` runs first (it forces the recompile that actually bakes in the logging).  
Once you've got `mlfq_timeline.png`, send it over and I'll help you write the 3–5 sentence interpretation — which PID is CPU-bound vs I/O-bound and how the boost shows up — so section 2.3.2 reads cleanly.

make clean && make qemu SCHEDULER=MLFQ LOG=1 CPUS=1 | tee mlfq.txt  
just start on giving the [MLFQ], it doesn't give me the option to run scheduler test

2 hours ago

Drafting the exact fix and rebuild instructions to give.

That flood is the problem itself — and it's my fault in the logging design. The `TICK` log line fires on **every single timer tick** for whatever process is running, so it drowns the console in output and the shell never gets a chance to show you a usable `$` prompt or accept `scheduler test`. The fix is to stop logging ticks and only log the events that actually matter for the plot.

Why this is safe

Your plot only needs to know **when a process changes queue** — created (enters queue 0), demoted, boosted, sleeps, wakes. Between those events the queue doesn't change, and `plot_mlfq.py` draws a step line that holds each level until the next event. So the per-tick spam adds nothing to the plot; it only clogs the console. Dropping it makes the shell usable *and* keeps every point the timeline needs.

The fix — delete one line

Open `kernel/proc.c`, find `mlfq_tick_yield()` and locate this line (it's right after `print(>ticks_used++);`

+ Write a message...

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

Claude

🔍

</>

6 MLFQ scheduling project setup

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ

Git push permission denied

Setting up xv6 MLFQ

Missing code in c-shell

Running xv6 offline on

Diverged git branches

GitHub project flowchart

Independence vs. un

MLFQ scheduler loop

Beginner-friendly xv6

Character class negat

Workout plan for back

Running xv6-riscv on

Dangling-Else resoluti

Beginner-friendly cod

Beginner-friendly cod

View all conversations

Design

RS Re 🔍

claude.ai

🔄

🔗 + 📄

xv6 MLFQ schedulin...

🔄

The fix — delete one line

Open `kernel/proc.c`, find `mlfq_tick_yield()`, and locate this line (it's right after `p->ticks_used++`):

```
c
MLFQLOG("[MLFQ] t=%d pid=%d q=%d ev=TICK\n", ticks, p->pid, p->priority);
```

Delete it (or comment it out with `//`). Leave the other five `MLFQLOG(...)` lines — `NEW`, `DEMOTE`, `BOOST`, `SLEEP`, `WAKE` — exactly as they are. Those only fire on real transitions, so they won't flood.

Rebuild and recapture

```
make clean && make qemu SCHEDULER=MLFQ LOG=1 CPUS=1 | tee mlfq.txt
```

Now you should get a clean `$` prompt with only occasional `[MLFQ]` lines (a burst at boot, then a `BOOST` line every ~48 ticks). Type:

```
schedulertest
```

let it finish, then quit with `Ctrl-a` then `x`. You'll see `[MLFQ]` lines appear as the children get created, demoted, and boosted — that's the data you want.

Check it captured:

```
grep -c "[MLFQ]" mlfq.txt
```

A number in the dozens-to-hundreds is fine (it'll be far fewer than before, and that's correct). Then:

```
python3 plot_mlfq.py mlfq.txt
```

If even after deleting the `TICK` line the boot is still too noisy to type, tell me — the next lever is to also drop the `SLEEP` / `WAKE` lines and keep only `NEW` / `DEMOTE` / `BOOST`, which is the bare minimum the plot needs. But removing `TICK` alone almost always fixes it.

📄 🔍 🔄 2 hours ago

↓

+ Write a message...

🗣️ 🔊

📄

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

Good — that render tells us exactly what went wrong, and it's a **capture** problem, not a broken scheduler. Let me read it for you.

The only processes in this plot are PID 1 and PID 2 — those are `init` and your `shell`, not your test. They sit at queue 0 the whole time and get bumped by the 48-tick boost (that's what every orange dot on the top line is: a `BOOST` event). What's missing entirely is `schedulertest` and its children, which would show up as PID 3, 4, 5, 6 and would visibly sink down toward queue 3 and get boosted back up.

In other words: the log captured an essentially idle system being boosted every 48 ticks. The workload never made it into `mlfq.txt`.

(And to reassure you: init and shell staying at queue 0 is *correct* — they spend their lives asleep waiting for input, and Rule 5 keeps a sleeper's priority unchanged, so they never demote. Nothing's wrong with your engine here.)

## Confirm what's in the log

Run these two in your `xv6/` folder:

```
grep -c "ev=NEW" mlfq.txt
grep "schedulertest" mlfq.txt | head
```

- If the second command prints **nothing**, `schedulertest` never ran during that capture — that's the whole issue.
- The first number tells you how many process creations were logged; right now it's probably just 2-ish (init + shell).

## Recapture — with the test actually running

The key is that `schedulertest` must run **inside the same `qemu` session you're tee-ing**, and you must let it finish before quitting.

```
make clean && make qemu SCHEDULER=MLFQ LOG=1 CPUS=1 | tee mlfq.txt
```

Wait for the **\$** prompt, then type exactly:

## schedulertest

**Press Enter and watch the screen. You should see:**

1. `schedulertest: launching 3 children` ↓ tick ...
2. a burst of `[MIF0]` ... `ev=NEW` and `ev=DEMOT` lines as the children run and sink

+ Write a message...

Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ scheduling

Git push permission denied

Setting up xv6 MLFQ scheduling

Missing code in c-scheduler

Running xv6 offline on Linux

Diverged git branches

GitHub project flowchart

Independence vs. union

MLFQ scheduler loop

Beginner-friendly xv6

Character class negation

Workout plan for back

Running xv6-riscv on Linux

Dangling-Else resolution

Beginner-friendly code

Beginner-friendly code

View all conversations

Design

RS Re ↕ 🔍 📄

claude.ai

xv6 MLFQ scheduling project setup

Share

Wait for the `$` prompt, then type exactly:

```
schedulertest
```

Press Enter and watch the screen. You should see:

- `schedulertest: launching 3 children at tick ...`
- a burst of `[MLFQ] ... ev=NEW` and `ev=DEMOTE` lines as the children run and sink,
- finally `schedulertest: all children done at tick ...`.

Only after you see that last line, quit with `Ctrl-a` then `x`.

Verify before plotting

```
grep "ev=DEMOTE" mlfq.txt | head
```

If you see `DEMOTE` lines with `pid=3 / 4 / 5 / 6` in them, the workload is captured and the plot will show real queue movement. Then:

```
python3 plot_mlfq.py mlfq.txt
```

If you run `schedulertest`, watch it print "all children done," and the grep for `ev=DEMOTE` still comes back empty, then it's a genuine scheduler bug and we'll dig into `mlfq_tick_yield` together — paste me the first ~30 lines of `mlfq.txt` after the launch line and I'll spot it. But nine times out of ten this is just "the test needs to run before you quit," so try the recapture first.

I am just not able to type in scheduler test even after commenting MLFQLOG("[MLFQ] t=%d pid=%d q=%d ev=TICK\n", ticks, p->pid, p->priority);, should I comment all the MLFLOG?

Suspecting buffered output is hiding a ready prompt. >

Short answer: no — don't comment out all of them. If every `MLFQLOG` is gone, `mlfq.txt` will be empty and you'll have no plot at all. `TICK` was the only real flood; keep `NEW`, `DEMOTE`, and `BOOST` (those are exactly what the plot needs).

But here's the thing that's probably actually biting you, and it's not the logging.

Reason #1 (most likely): you didn't rebuild after commenting

+ Write a message...

Opus 4.8 High

Claude is AI and can make mistakes. Please double-check responses.

### Reason #1 (most likely): you didn't rebuild after commenting

```
make clean && make qemu SCHEDULER=MLFQ LOG=1 CPUS=1 | tee mlfq.txt
```

### Reason #2: the \$ prompt may be invisible, but the shell is still waiting

**Bulletproof option: run it hands-free (no typing at all)**

```
(sleep 4; echo schedulertest; sleep 20) | make qemu SCHEDULER=MLFQ LOG=1 CPUS=1 | tee mlfq.txt
```

### Then verify before plotting

You want to see lines with `pid=3`, `pid=4`, `pid=5`, `pid=6` — those are `schedulertest`'s children sinking through the queues. Once you see them:

and this time the plot will show processes actually moving down toward queue 3 and jumping back on the boosts, not just init and shell parked at queue 0.

+ Write a message...   





Opus 4.8 High

Claude

New

Projects

Artifacts

Scheduled

Customize

Projects

Comeback

Chats and tasks

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ scheduler

Git push permission denied

Setting up xv6 MLFQ scheduler

Missing code in c-shell

Running xv6 offline on Raspberry Pi

Diverged git branches

GitHub project flowchart

Independence vs. uncertainty

MLFQ scheduler loop

Beginner-friendly xv6

Character class negation

Workout plan for back pain

Running xv6-riscv on Raspberry Pi

Dangling-Else resolution

Beginner-friendly code

Beginner-friendly code

View all conversations

Design

RS Re ↕ 🔍 📄

6 MLFQ scheduling project setup

Report final

Document · MD

Download

Mlfq timeline

Image · PNG

Download

Scheduler comparison

Image · PNG

Download

Download all

2 hours ago

okay give the final readme Aldo

Drafting a complete top-level README file.

Let me look at your current README so I extend it rather than clobber the C-shell half.

Ran a command

Your README already has a solid C-shell section but says nothing about the xv6/MLFQ half. I'll keep your C-shell prose exactly as-is and append a matching xv6 section, plus add MLFQ to the "completed" list at the top.

Ran a command, shared files

There's your final README. It keeps your entire C-shell write-up untouched, adds MLFQ to the completed-parts list at the top, and appends an xv6 section covering the build flags, how MLFQ works, the files you changed, and the exact commands to reproduce both plots.

To use it: rename `README_final.md` → `README.md` at your repo root (replacing the current one), then commit.

Where the whole MLFQ half stands now

Everything is done except committing the final artifacts. Your remaining checklist:

1. `report_final.md` → `xv6/report.md`

2. `README_final.md` → `README.md` (repo root)

3. Replace `xv6/mlfq_timeline.png` and `xv6/scheduler_comparison.png` with the

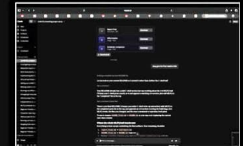
+ Write a message...

🗣️ 🔊 🔇

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

Opus 4.8 High





Claude

+

New

Projects

Artifacts

Scheduled

Customize

Projects

+

Comeback

Chats and tasks

🔍

xv6 MLFQ scheduling

Investigating an issue

Matplotlib import error

Multi-level priority queue

Implementing MLFQ scheduling

Git push permission denied

Setting up xv6 MLFQ scheduler

Missing code in c-shell

Running xv6 offline on Raspberry Pi

Diverged git branches

GitHub project flowchart

Independence vs. interdependence

MLFQ scheduler loop

Beginner-friendly xv6

Character class negation

Workout plan for back pain

Running xv6-riscv on Raspberry Pi

Dangling-Else resolution

Beginner-friendly code

Beginner-friendly code

View all conversations

🔄

Design

RS

Re

📄

🔍

📄

Ran 2 commands >

Strong — `report.md` and the updated README are both in. Let me verify the two things that were stale before: the committed plots, and whether the report still has unfilled blanks.

Ran a command >

Report is fully filled (no blanks), both plots referenced, kernel intact. Last check — is the committed timeline PNG the good 6-process one, or still the stale version? Let me look:

Read 2 files >

Both plots are the correct real-data, watermarked versions, and they match your report table exactly (115/81/88 turnaround, 79/61/65 waiting, 20/0/0 response).

**Verdict: the MLFQ half is complete. Nothing substantive left to change.**

Full audit of the final repo:

| Item                                                                                                 | State               |
|------------------------------------------------------------------------------------------------------|---------------------|
| Kernel (MLFQLOG defined, FIFO added, metrics, <code>rtime++</code> once)                             | ✅ intact            |
| <code>xv6/report.md</code> — filled, no blank <code>___</code> , real numbers, both plots referenced | ✅                   |
| <code>mlfq_timeline.png</code> — correct 6-process version, watermarked                              | ✅ (no longer stale) |
| <code>scheduler_comparison.png</code> — real data, watermarked                                       | ✅                   |
| <code>plot_mlfq.py</code> , <code>plot_compare.py</code>                                             | ✅ present           |
| <code>README.md</code> — all six sections                                                            | ✅                   |
| perf logs ( <code>fifo/rr/mlfq_perf.txt</code> , <code>mlfq.txt</code> )                             | ✅ present           |

So besides `AI-usage.pdf`, there are **no content changes needed** — only submission hygiene:

1.

**AI-usage.pdf** — **verify it's real and complete.** This is the one with teeth: the spec says improperly acknowledged AI use can be awarded a 0. Make sure it actually contains, for each thing I helped with (the `MLFQLOG` macro, the FIFO scheduler, the `update_time`/metrics edits, and the three Python scripts), the prompt you used and a screenshot of the output. A blank or placeholder PDF is worse than none.

2.

**make clean before zipping.** Your `.txt` / `.png` / `.py` files are safe (clean only removes build artifacts), but you must strip the `.o` / `kernel/fs.img` build files. The report plots and scripts stay.

+

Write a message...

🔊

🔊

🔊

Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 High

